AWS VPC NETWORKING

Public and Private Subnet Deployment

Step-by-step practical guide with console screenshots

Lab component	Purpose
Custom VPC	Isolated network for all resources in this exercise
Public subnet + Internet Gateway	Hosts the internet-facing Home Page EC2 instance
Private subnet + NAT Gateway	Hosts the private Login Page EC2 instance; outbound-only internet path
Route tables + security groups	Control network paths and permitted traffic
Two EC2 instances	Demonstrate public and private application access

1. Lab overview and network design

The lab builds a dedicated Amazon Virtual Private Cloud (VPC) with two subnets. The public subnet contains an EC2 instance that serves a Home Page to internet users. The private subnet contains a second EC2 instance that serves a Login Page and has no public IPv4 address.

Traffic design:

Traffic	Expected path
Internet user → Home Page	Internet → Internet Gateway → Public subnet → Public EC2 (TCP 80)
Public EC2 → Login Page	Public EC2 → Private subnet → Private EC2 (TCP 80), permitted by security-group reference
Private EC2 → Internet (outbound)	Private EC2 → NAT Gateway in public subnet → Internet Gateway → Internet
Internet → Private EC2	Not permitted directly; the private instance has no public IPv4 address

2. Addressing plan

Resource	Example name	CIDR / setting
VPC	srinivas-vpc	17.0.0.0/16
Public subnet	srinivas-public	17.0.10.0/24
Private subnet	srinivas-private	17.0.20.0/24
Public route table	srinivas-public-route	0.0.0.0/0 → Internet Gateway
Private route table	srinivas-private-route	0.0.0.0/0 → NAT Gateway
Public EC2	Srinivas-Homepage	Public IPv4 enabled
Private EC2	Srinivas-LoginPage	Public IPv4 disabled

The screenshots show the resource names used during the console exercise. Use the CIDR values in the addressing plan consistently; verify the actual values in your account before launching instances.

3. Create the VPC

Open AWS Management Console → VPC → Your VPCs → Create VPC.

1. Choose “VPC only” under Resources to create.
2. Enter a name such as srinivas-vpc.
3. Set IPv4 CIDR to 17.0.0.0/16 and tenancy to Default.
4. Choose Create VPC. Open the VPC details and confirm that its state is Available.

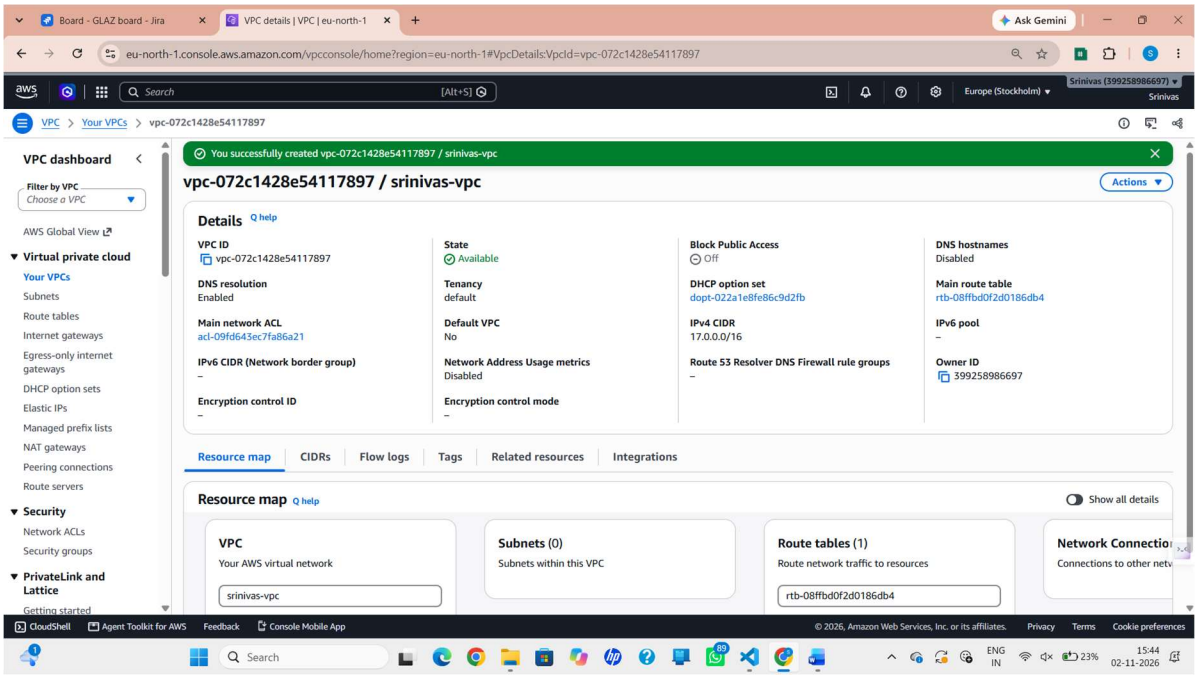


Figure 1. New VPC details page.

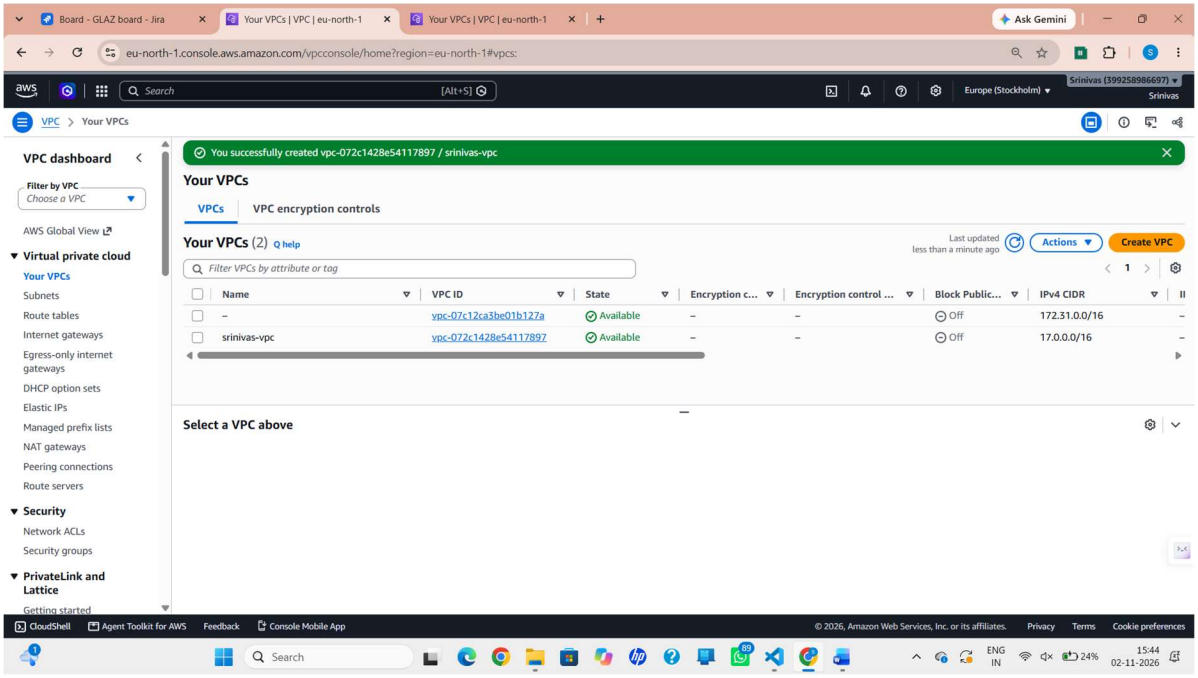


Figure 2. Your VPCs list showing the created VPC.

4. Create public and private subnets

Open VPC → Subnets → Create subnet. Select the newly created VPC for both subnet resources.

Setting	Public subnet	Private subnet
Name	srinivas-public	srinivas-private
VPC	Newly created VPC	Same newly created VPC
IPv4 CIDR	17.0.10.0/24	17.0.20.0/24
Availability Zone	Choose an available AZ	Preferably a different AZ

5. Create the public subnet with its name, Availability Zone, and CIDR.
6. Repeat the process for the private subnet, ensuring that both subnet CIDRs are within the VPC and do not overlap.
7. Open the Subnets list and verify that both subnets belong to the new VPC.

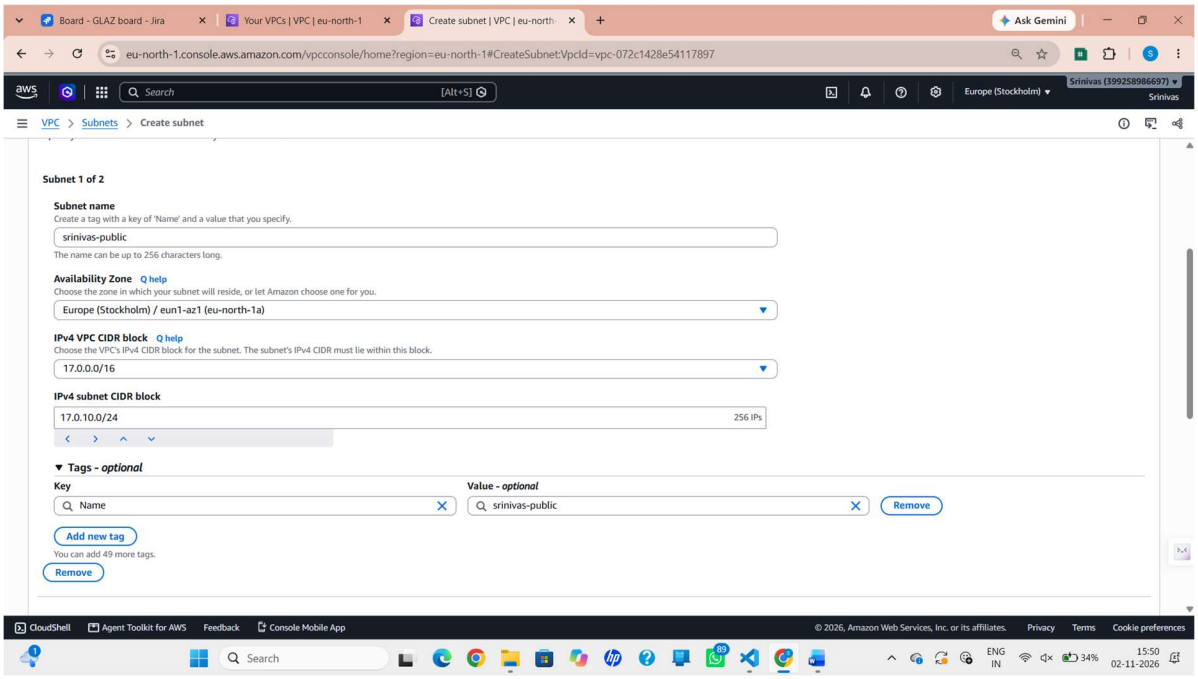


Figure 3. Subnet creation form – first subnet.

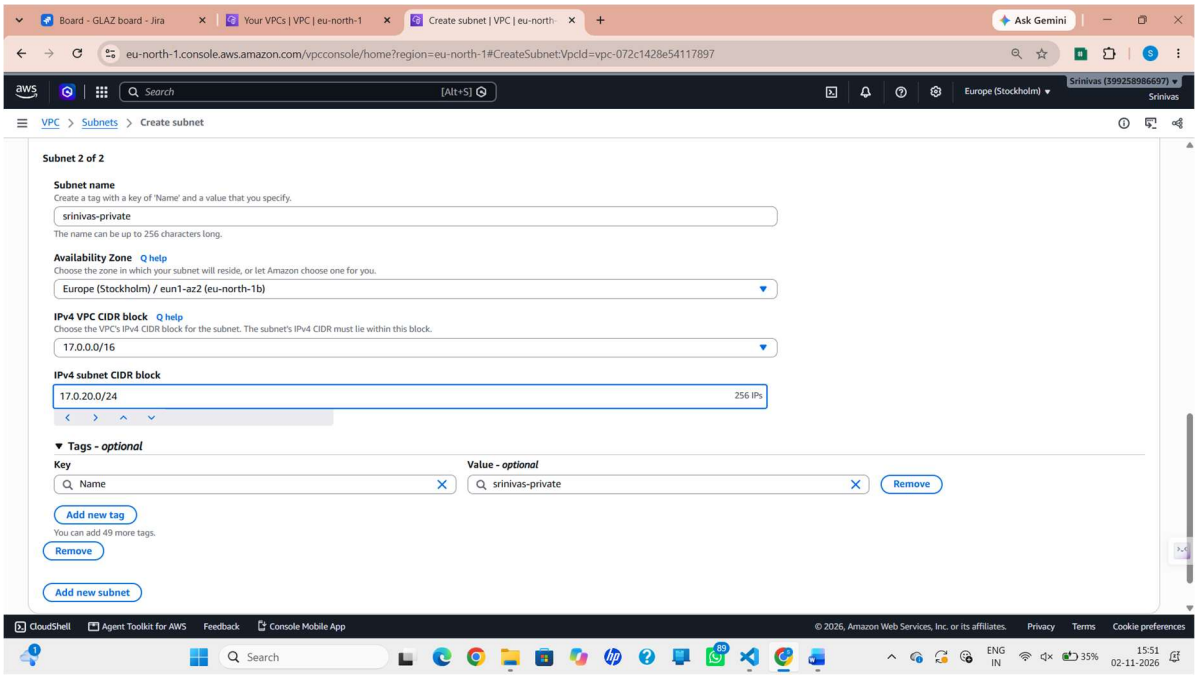


Figure 4. Subnet creation form – second subnet.

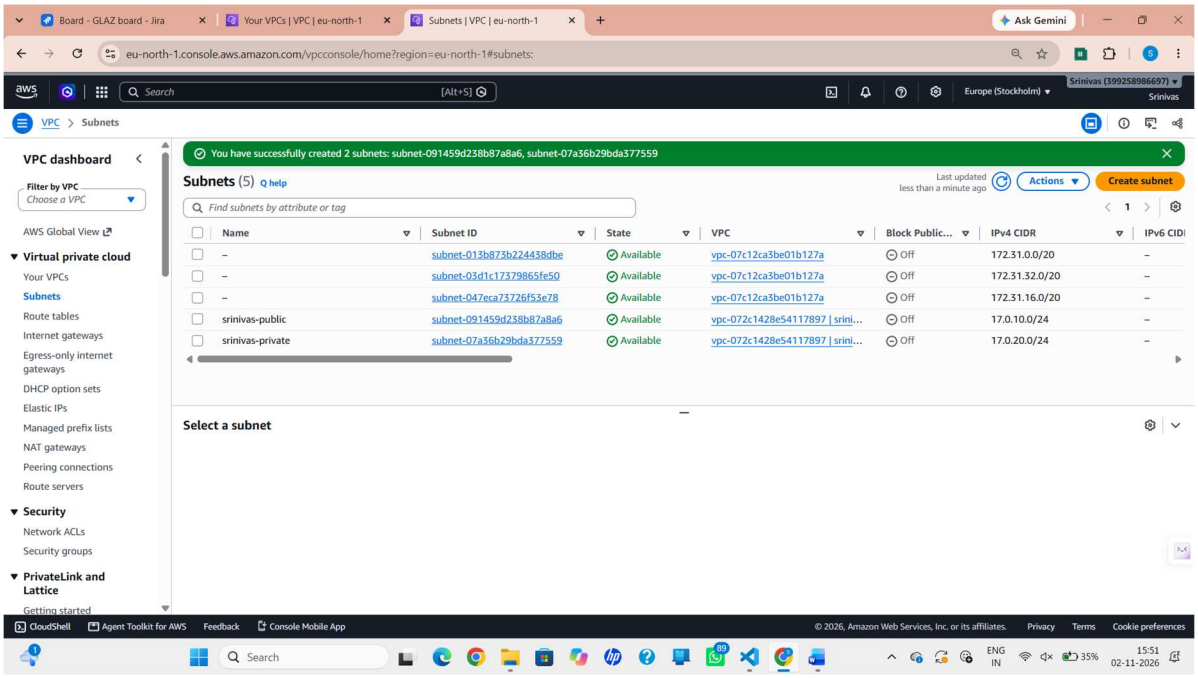


Figure 5. Subnets list showing the created subnet resources.

5. Create and attach an Internet Gateway

Open VPC → Internet Gateways → Create internet gateway. Enter a name such as srinivas-igw and create it.

8. Select the new Internet Gateway, choose Actions → Attach to a VPC, and select the VPC created for this lab.
9. Confirm that the gateway state is Attached.

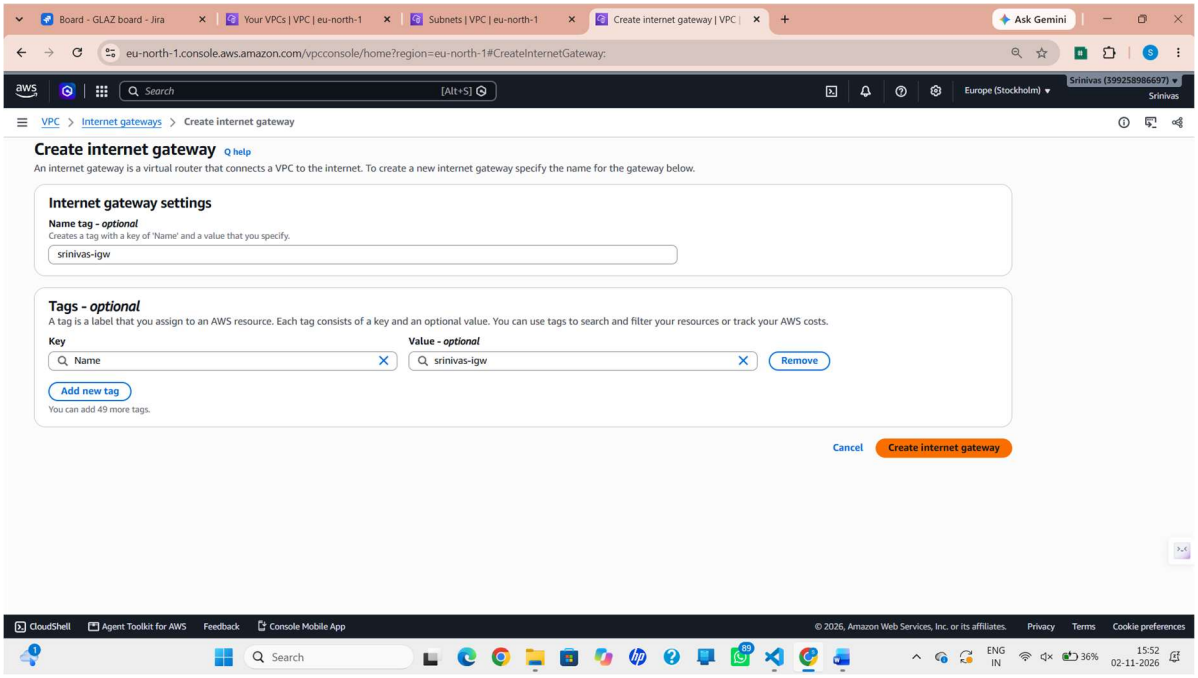


Figure 6. Create Internet Gateway form.

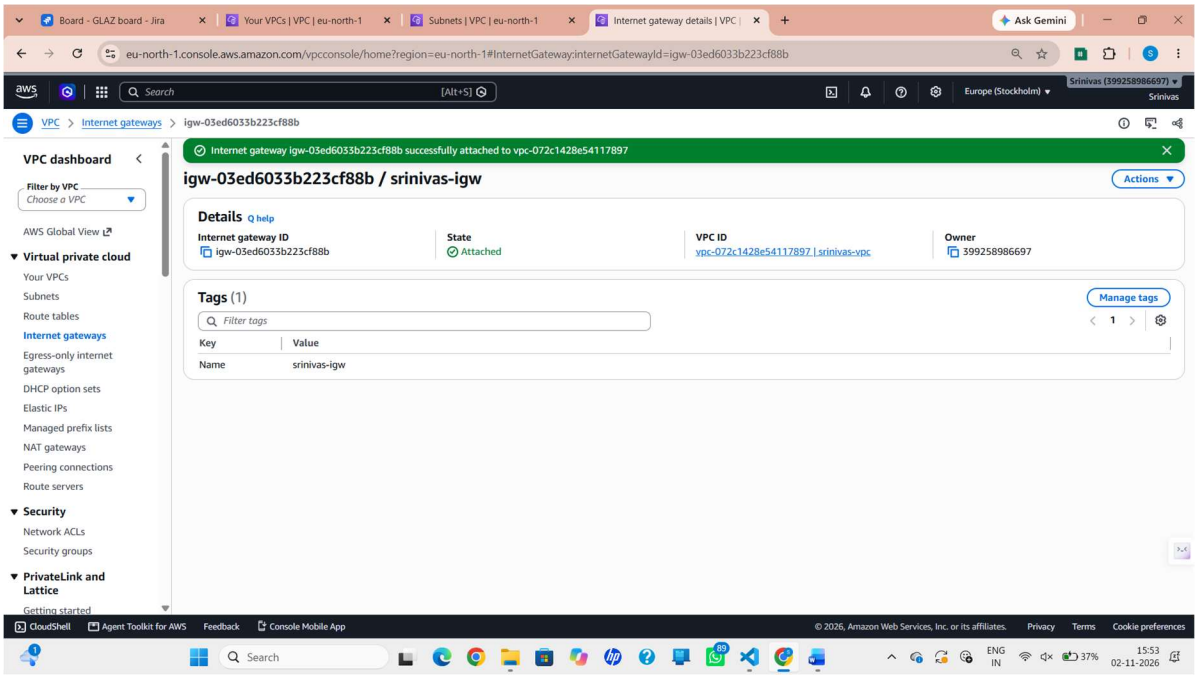


Figure 7. Internet Gateway details showing its attached VPC.

6. Configure the public route table

Open VPC → Route Tables → Create route table. Name it srinivas-public-route and select the new VPC.

10. Open the Routes tab and choose Edit routes.
11. Keep the existing local route for 17.0.0.0/16. Add a route with destination 0.0.0.0/0 and target the Internet Gateway created above.
12. Open Subnet associations → Edit subnet associations. Select only the public subnet and save.

Destination	Target	Purpose
17.0.0.0/16	local	Communication within the VPC
0.0.0.0/0	Internet Gateway	Internet route for public subnet

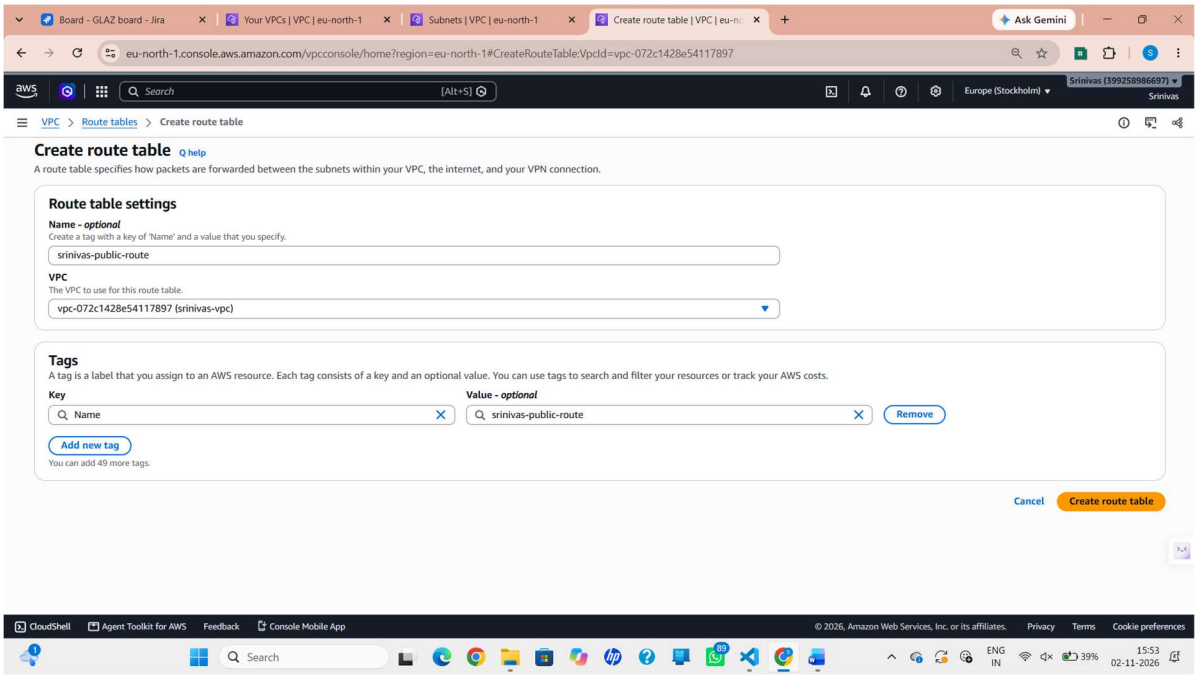


Figure 8. Create public route table.

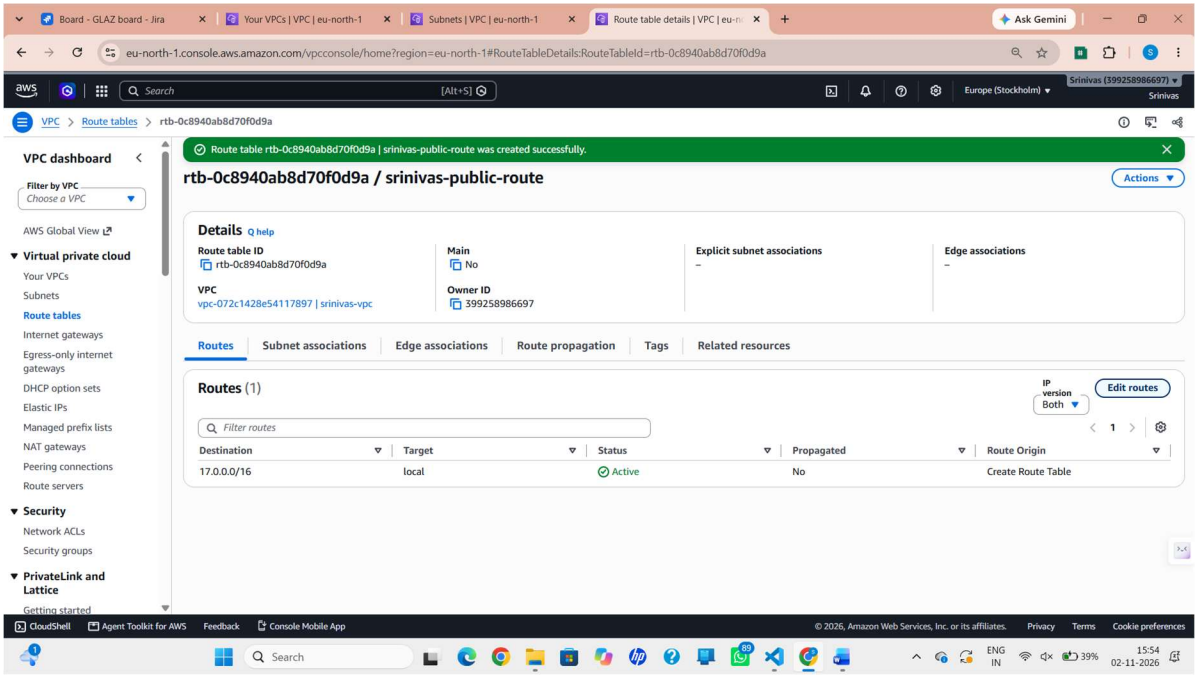


Figure 9. Public route table details.

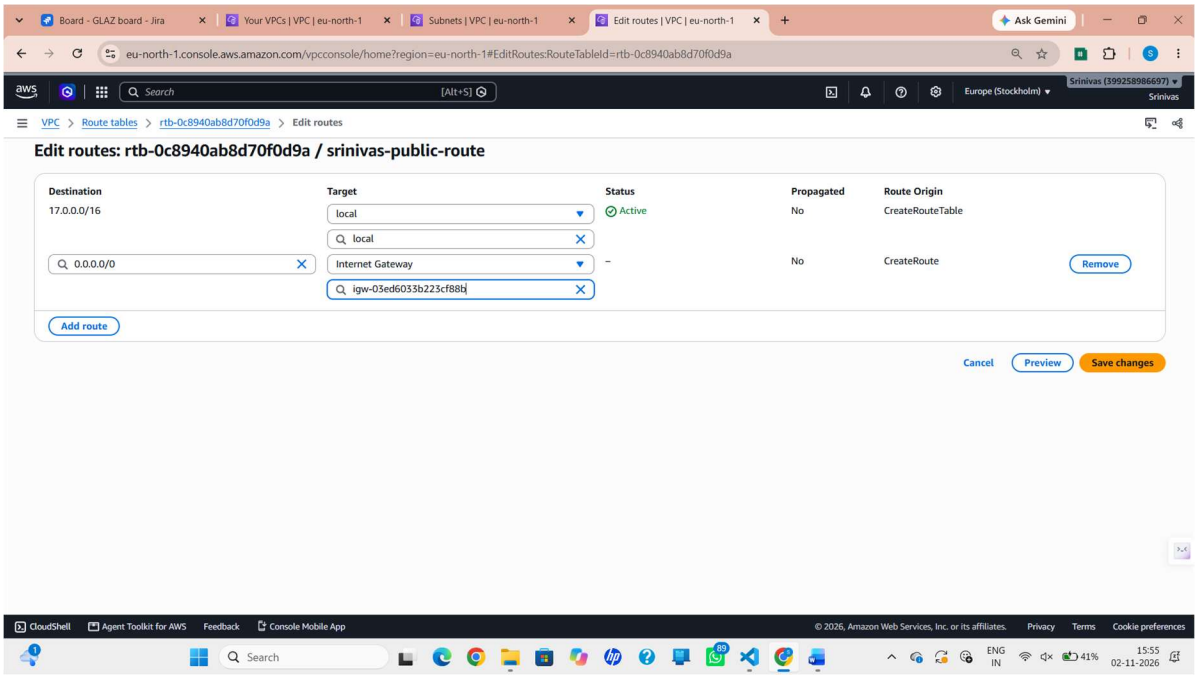


Figure 10. Edit routes and add the default route to the Internet Gateway.

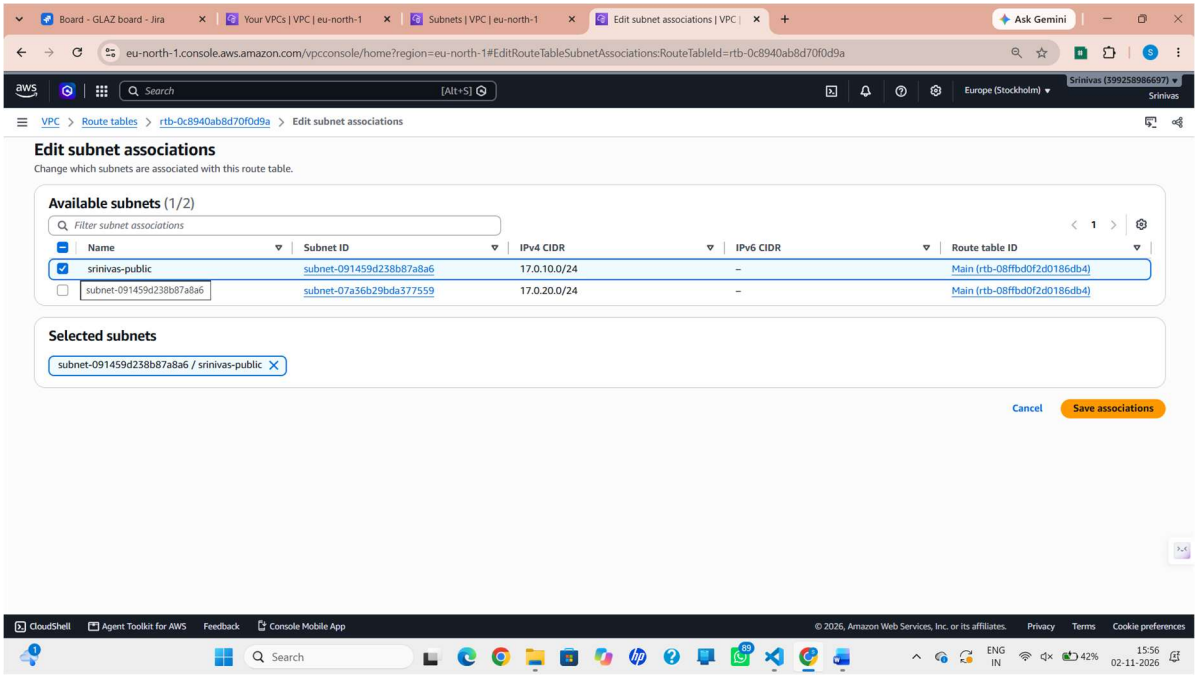


Figure 11. Associate the public subnet with the public route table.

7. Create a NAT Gateway

Open VPC → NAT Gateways → Create NAT gateway. The NAT Gateway must be placed in the public subnet.

13. Enter a name such as srinivas-nat.
14. Select the public subnet.
15. Choose Public connectivity (zonal).
16. Allocate a new Elastic IP address.
17. Create the NAT Gateway and wait until its status becomes Available.

A NAT Gateway allows instances in the private subnet to initiate outbound connections. It does not provide unsolicited inbound internet access to the private instance.

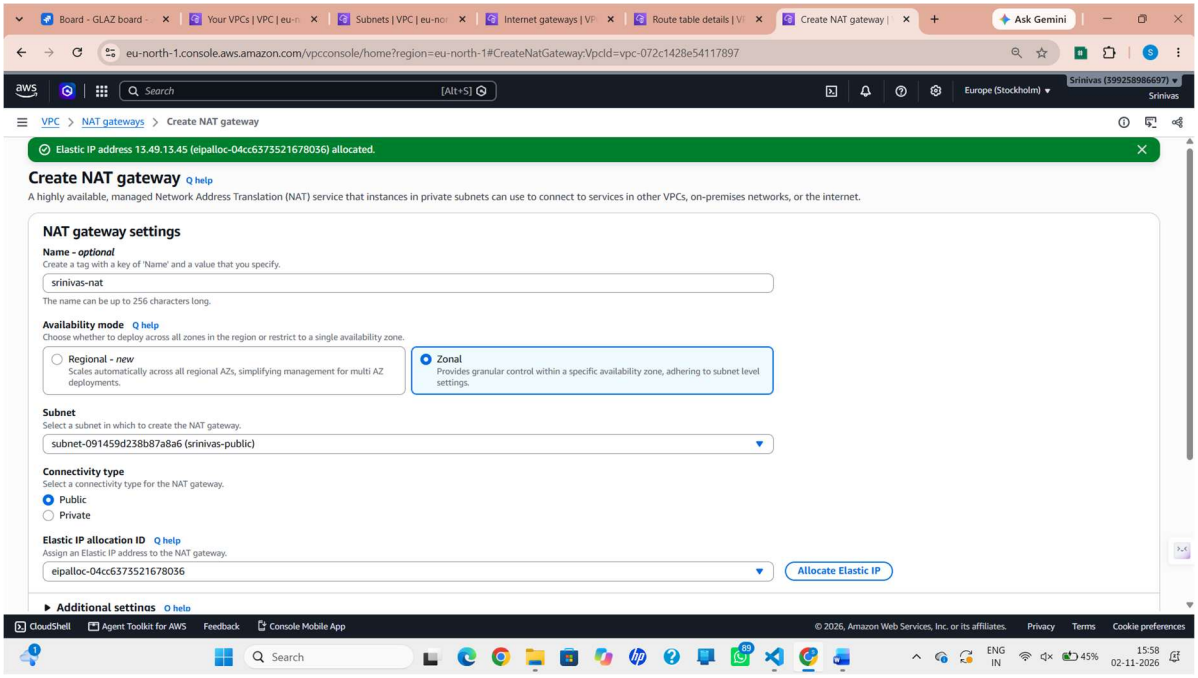


Figure 12. NAT Gateway creation settings.

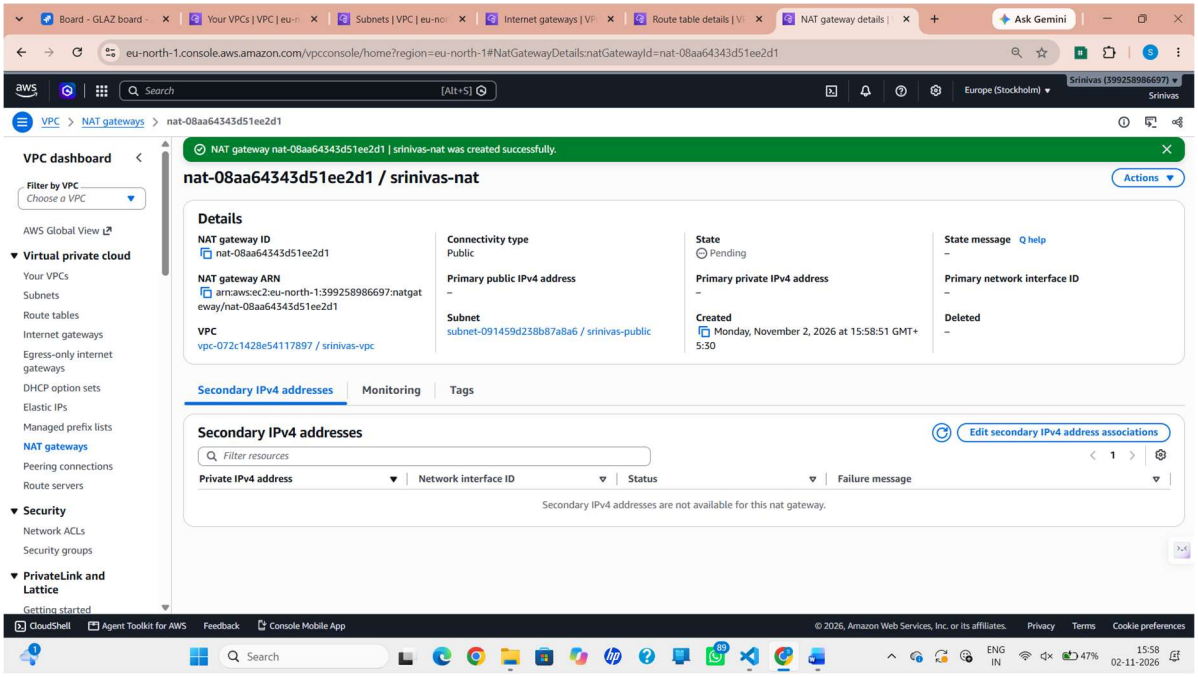


Figure 13. NAT Gateway resource details while provisioning.

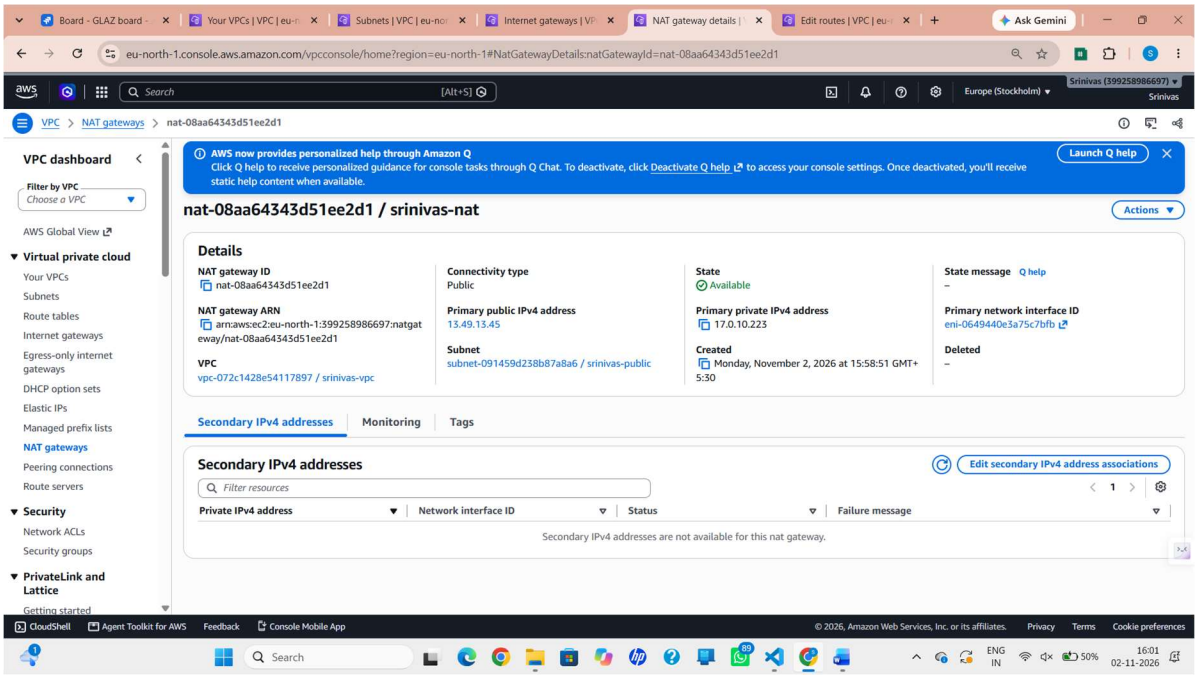


Figure 14. NAT Gateway showing Available status and its public address.

8. Configure the private route table

Create a second route table named srinivas-private-route in the same VPC.

18. Open Routes → Edit routes. Keep the local route 17.0.0.0/16.
19. Add destination 0.0.0.0/0 and select the NAT Gateway as the target.
20. Open Subnet associations → Edit subnet associations. Select the private subnet and save.

Destination	Target	Purpose
17.0.0.0/16	local	Communication within the VPC
0.0.0.0/0	NAT Gateway	Outbound internet access for private subnet

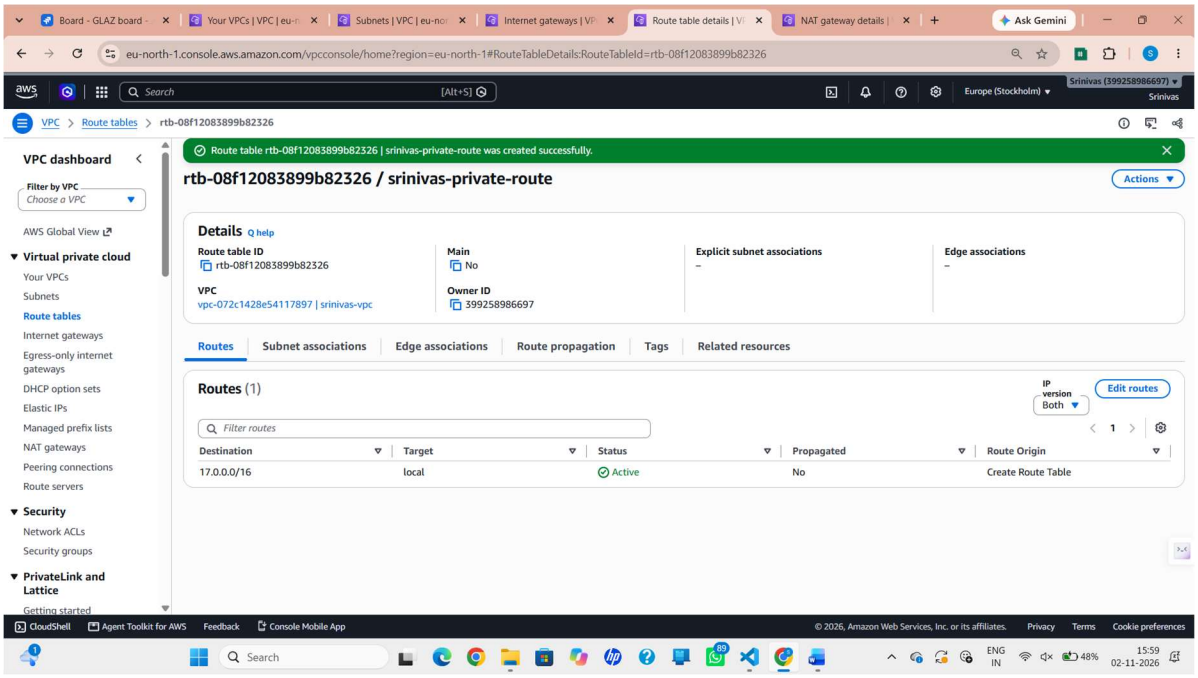


Figure 15. Private route table creation / configuration.

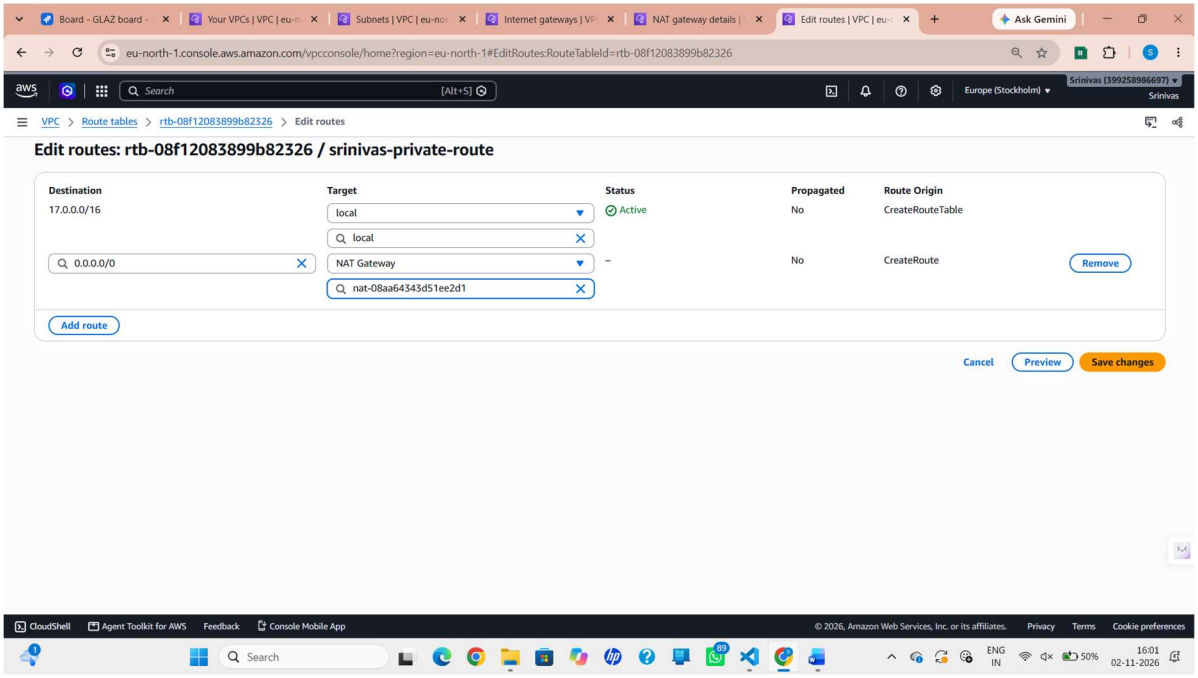


Figure 16. Private route table details.

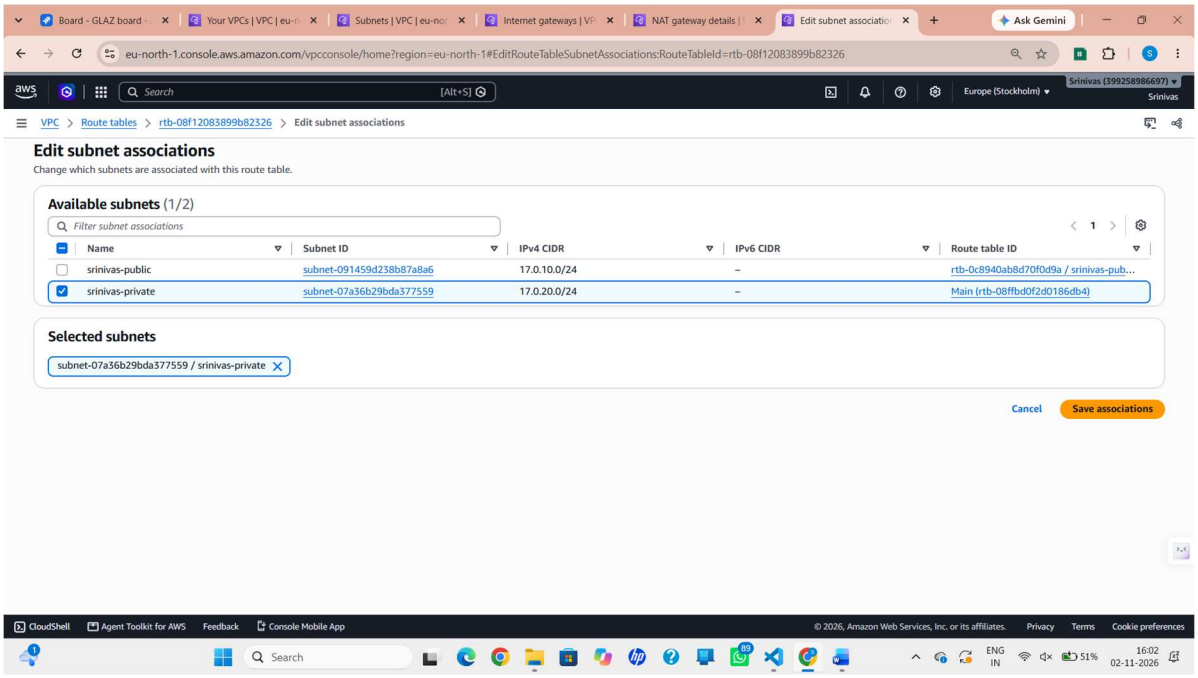


Figure 17. Edit private route to use the NAT Gateway.

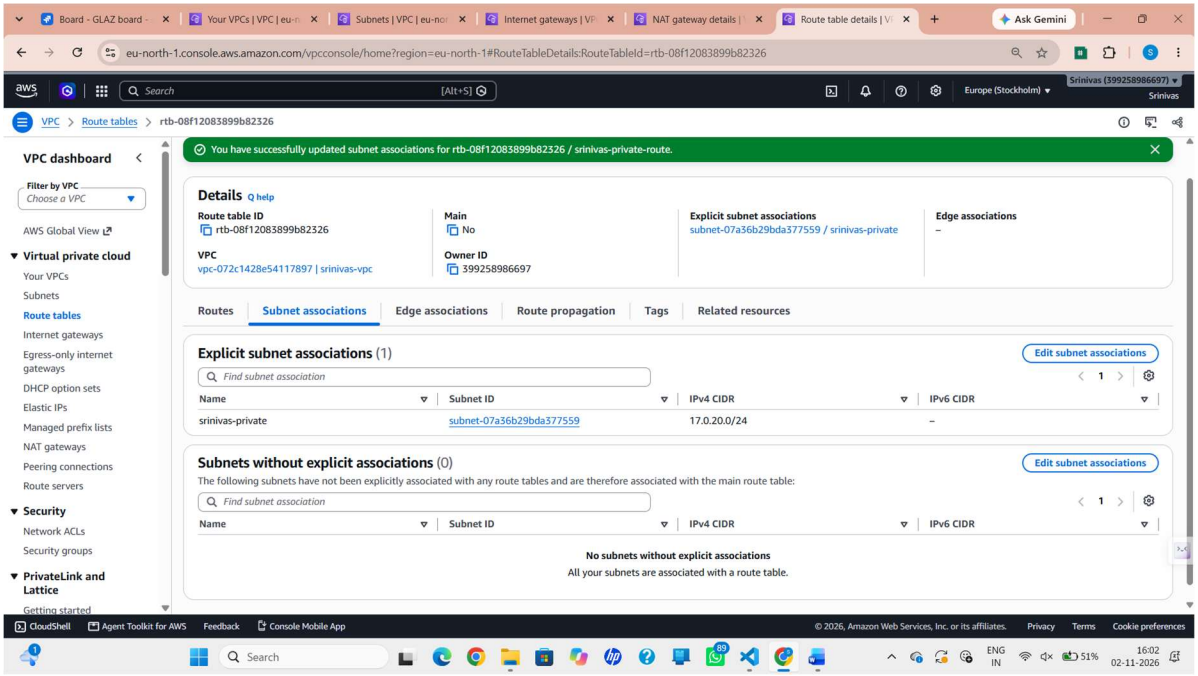


Figure 18. Associate the private subnet with the private route table.

9. Create security groups

Open EC2 → Security Groups → Create security group. Ensure the selected VPC is the newly created VPC, not the default VPC.

9.1 Public security group

Create a group such as srinivas-public-sg and configure these inbound rules:

Type	Protocol	Port	Source
SSH	TCP	22	Your public IP (/32) only

HTTP	TCP	80	0.0.0.0/0
HTTPS	TCP	443	0.0.0.0/0

9.2 Private security group

Create a group such as srinivas-private-sg. Allow HTTP from the public security group ID. If SSH administration is needed, allow TCP 22 from the public security group ID only.

Type	Protocol	Port	Source
HTTP	TCP	80	Public security group ID
SSH (optional)	TCP	22	Public security group ID

Do not use 0.0.0.0/0 as the source for private-instance HTTP or SSH. A security-group reference permits traffic from network interfaces associated with the referenced security group, subject to routing and network ACLs.

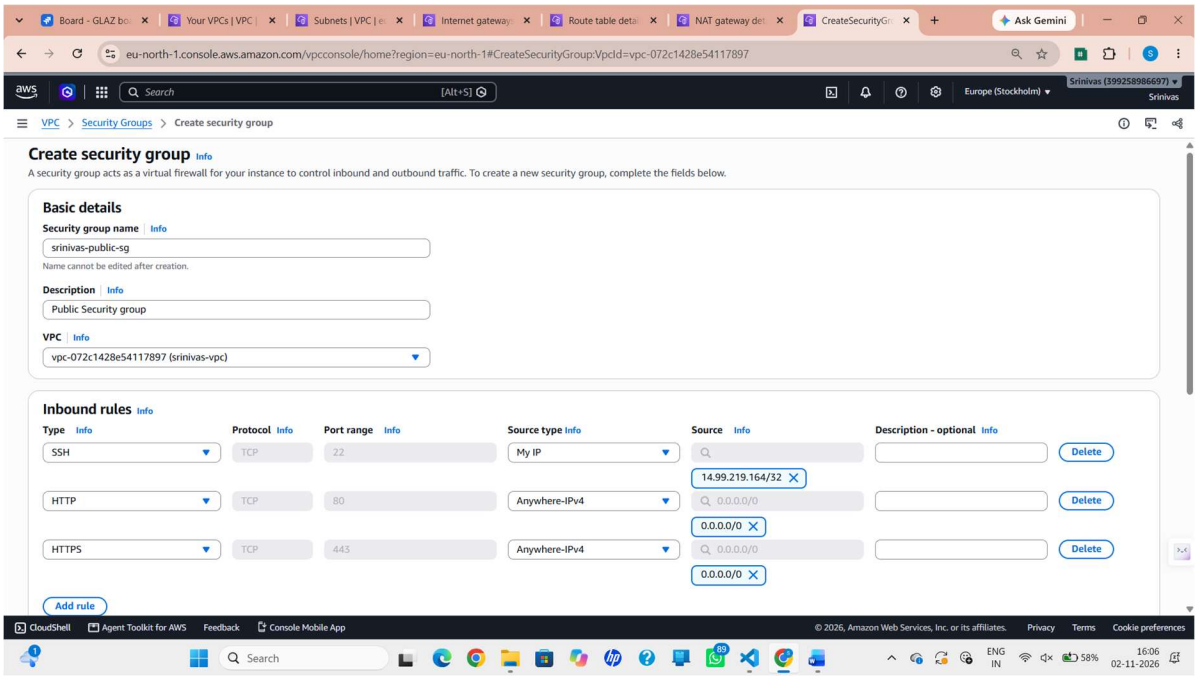


Figure 19. Create the public security group and configure inbound rules.

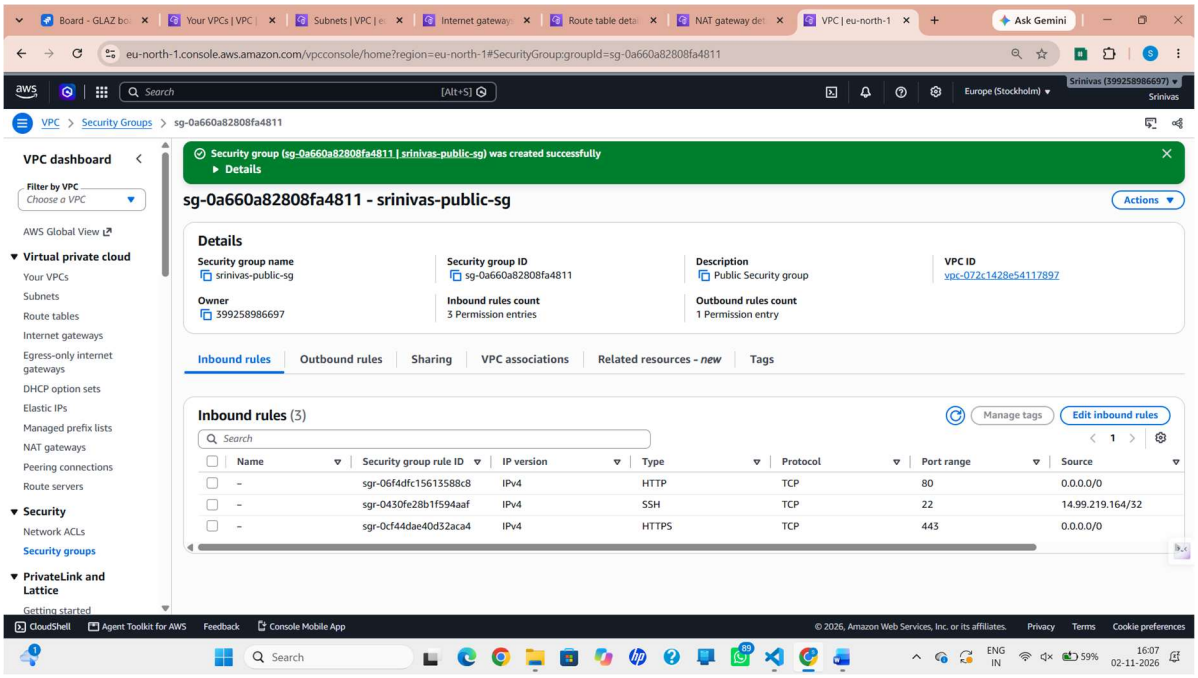


Figure 20. Public security group created.

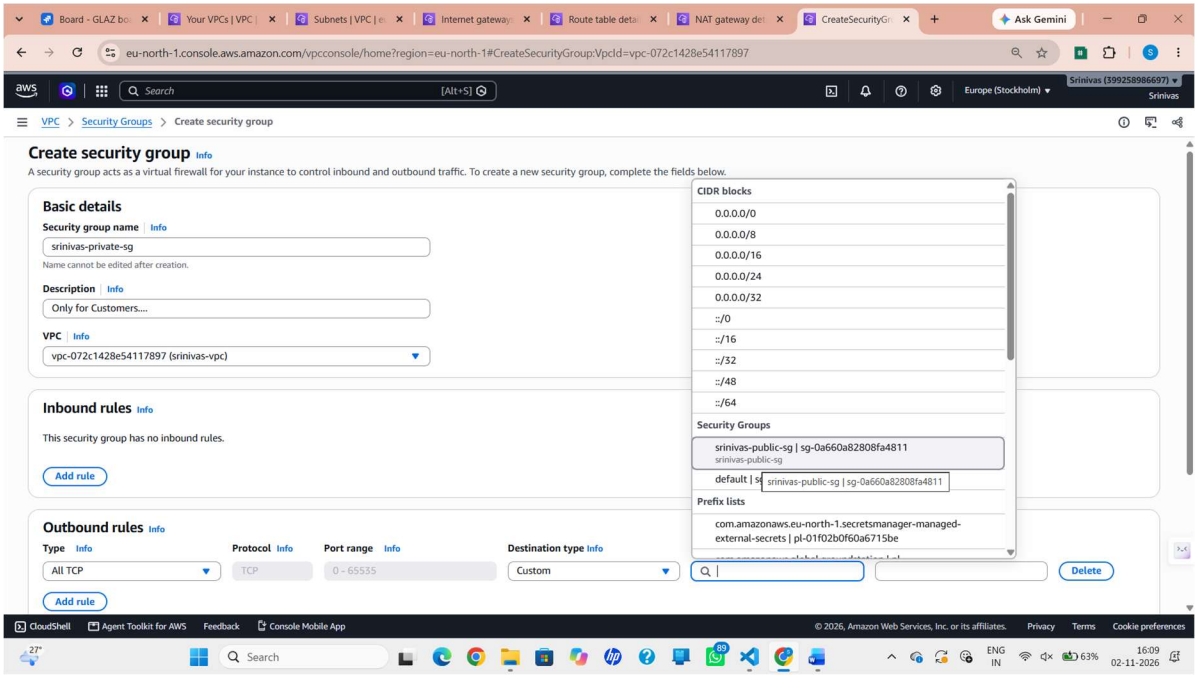


Figure 21. Choose the public security group as the source for the private security group rule.

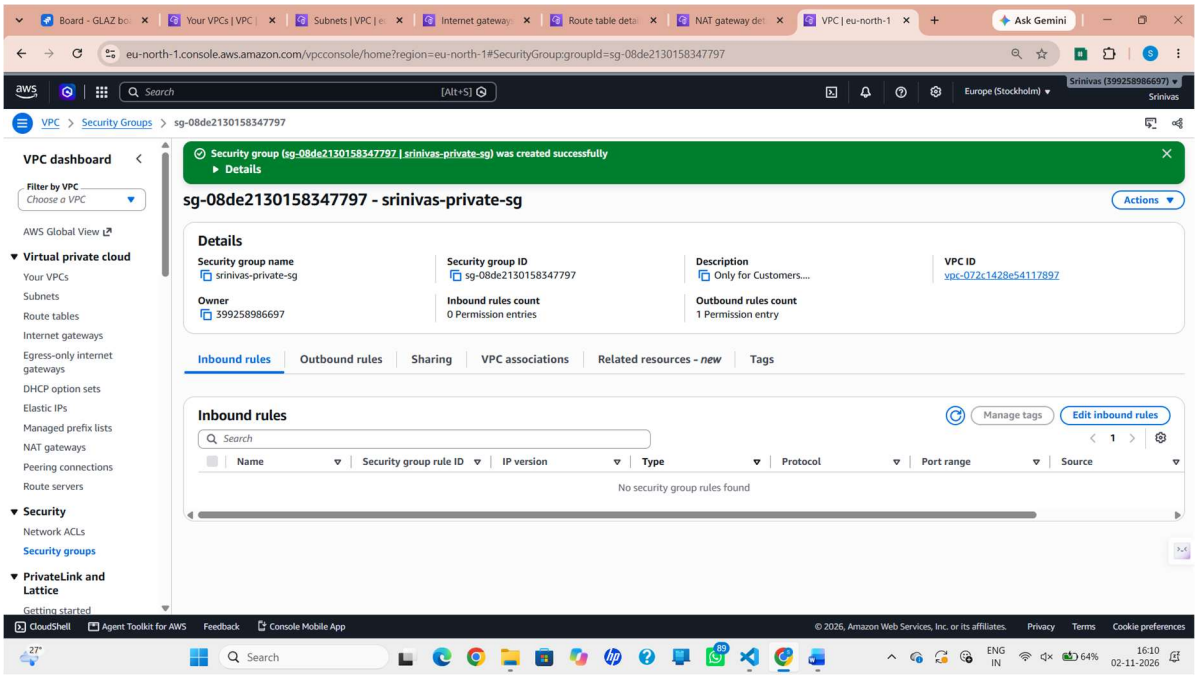


Figure 22. Private security group details.

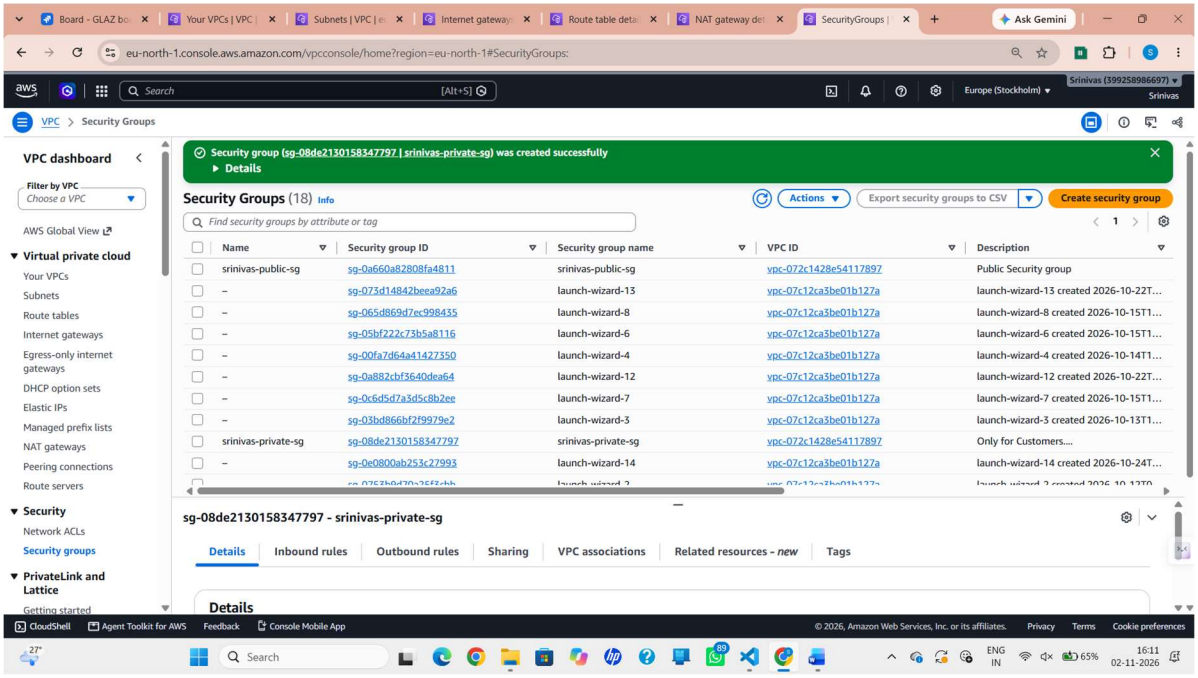


Figure 23. Security groups list and selected group details.

10. Launch the public EC2 instance

Open EC2 → Instances → Launch instances.

21. Enter an instance name such as Srinivas-Homepage.
22. Choose Ubuntu Server or Amazon Linux and a suitable instance type.
23. Select or create an SSH key pair and store the private key securely.
24. Under Network settings, choose the new VPC and public subnet.
25. Enable Auto-assign public IP. Select the public security group.
26. Launch the instance and wait for the instance state and status checks to become healthy.

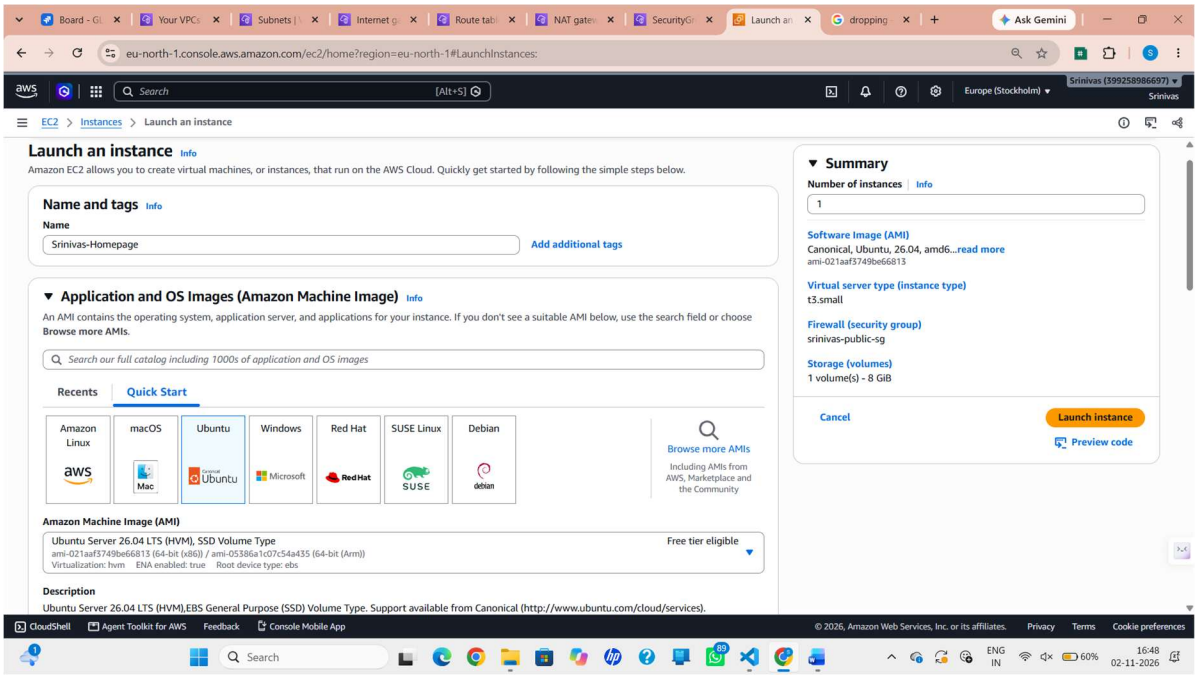


Figure 24. Launch instance – name, AMI, and instance configuration.

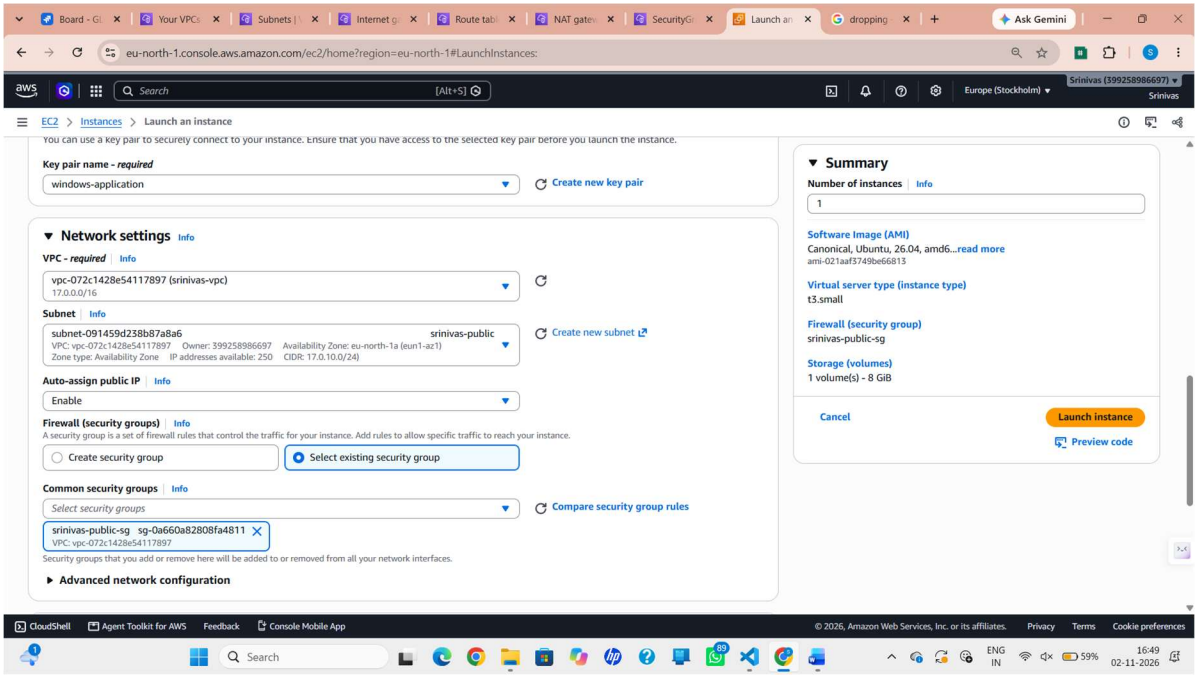


Figure 25. Launch instance – network settings for the public subnet.

11. Launch the private EC2 instance

Launch a second instance using the same or a suitable Linux AMI.

27. Enter a name such as Srinivas-LoginPage.
28. Choose a key pair suitable for secure administration. Do not copy a private key onto the public server.
29. Select the new VPC and the private subnet.
30. Disable Auto-assign public IP.
31. Attach only the private security group.
32. Launch the instance and verify that it has no public IPv4 address.

The supplied launch screenshots show the EC2 launch workflow and network/security-group selection. Confirm the final instance subnet and public-IP setting in the EC2 instance details.

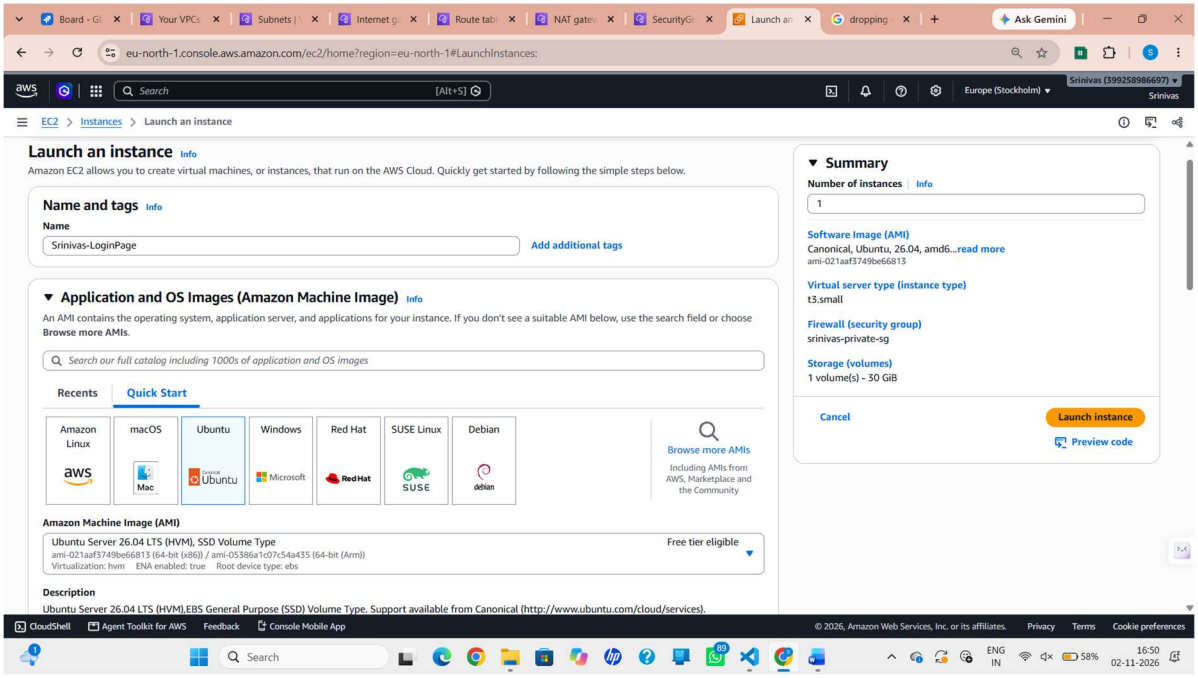


Figure 26. Review public EC2 launch configuration.

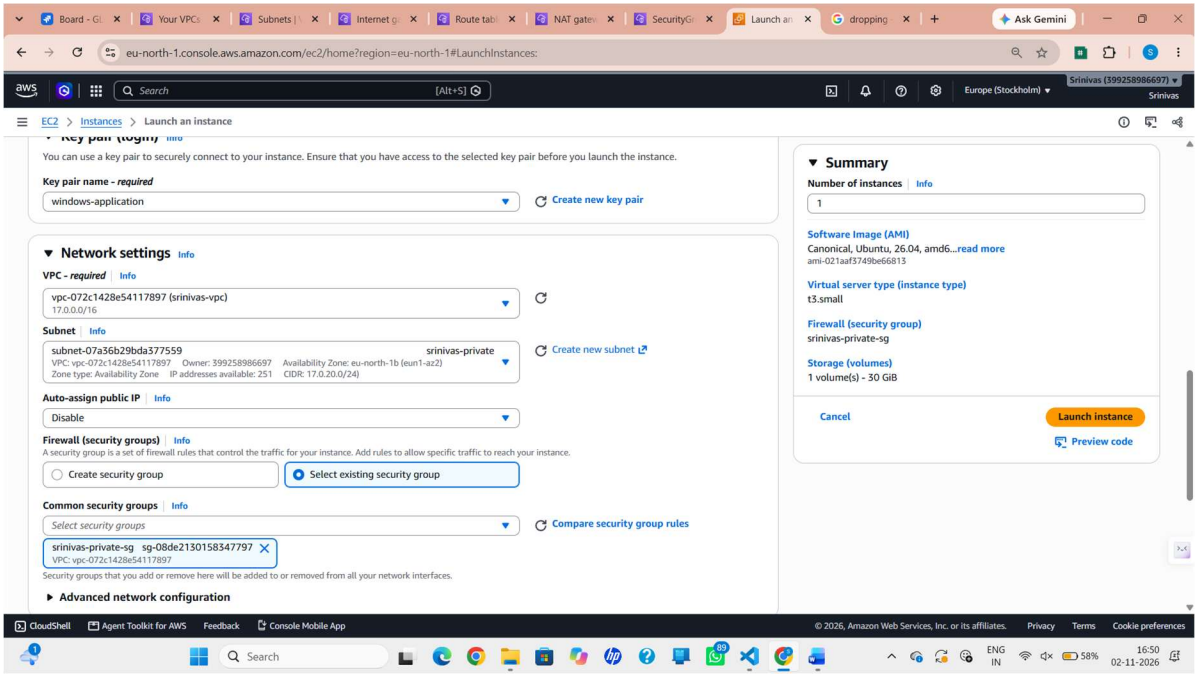


Figure 27. Launch instance configuration and security group selection.

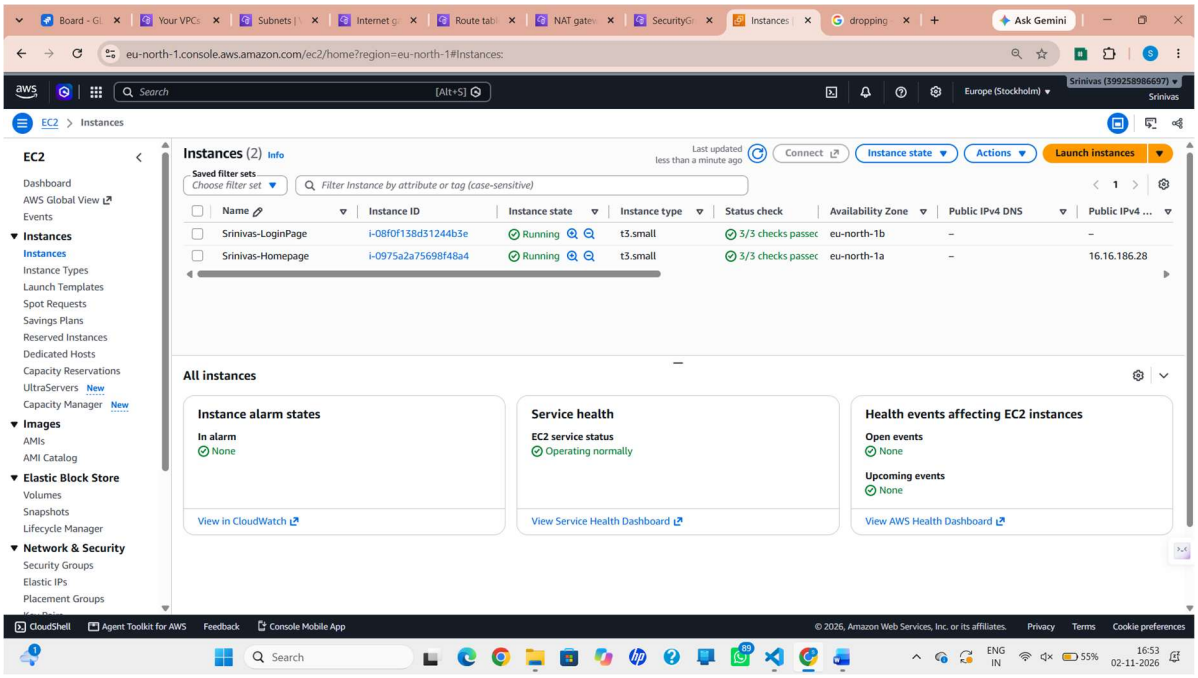


Figure 28. EC2 instances list showing launched instances.

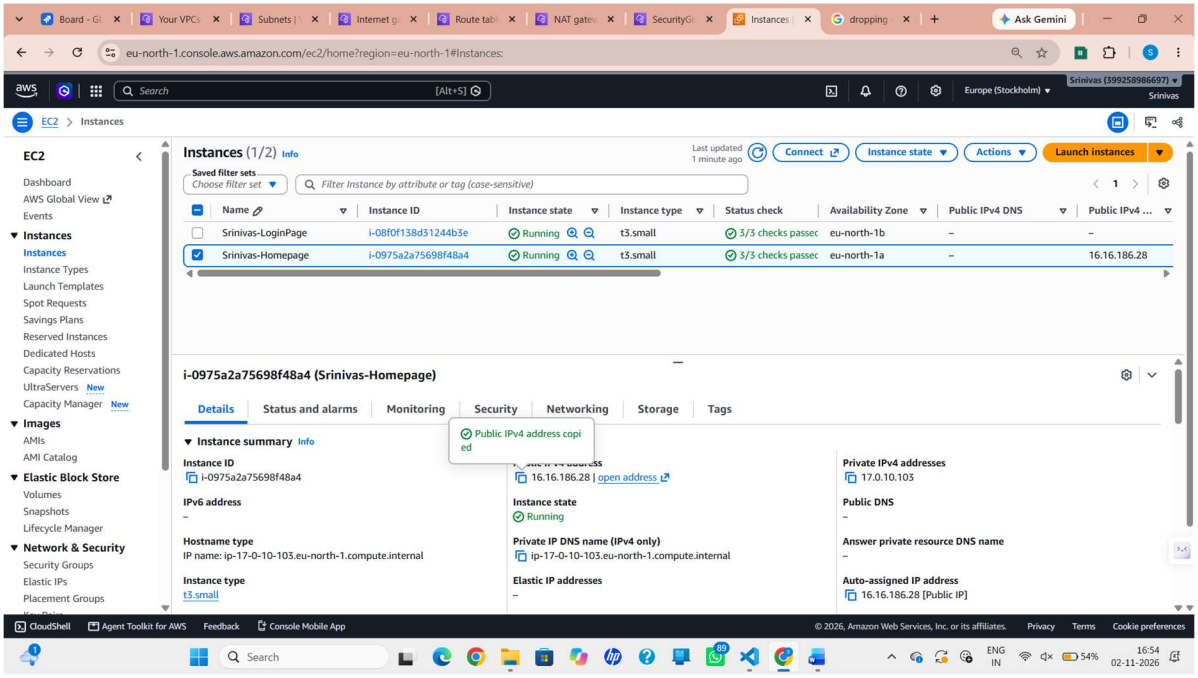


Figure 29. Public EC2 instance details and assigned public IPv4 address.

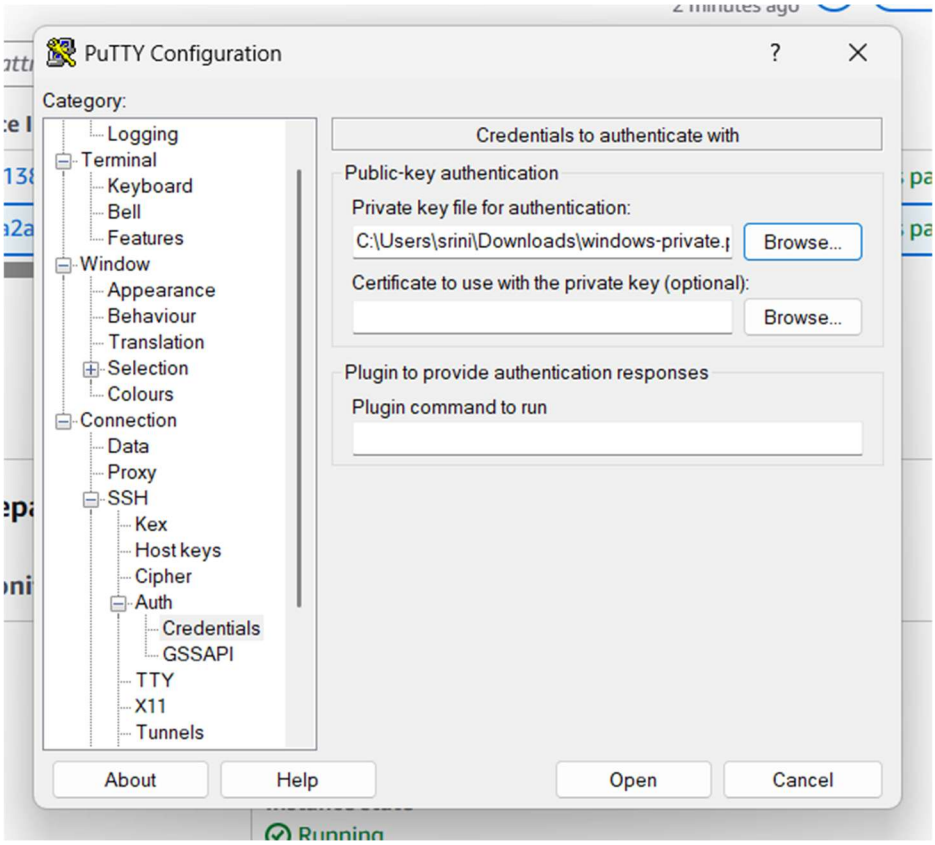


Figure 30. SSH client configuration for connecting to the instance.

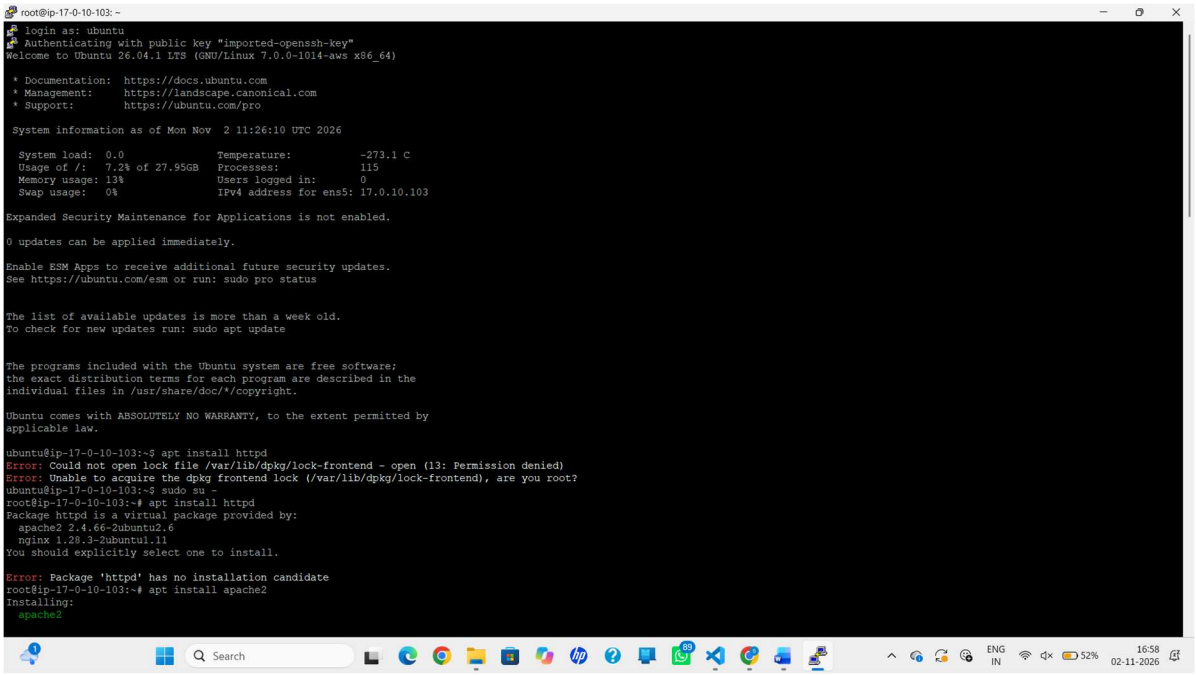


Figure 31. Terminal session and Linux server setup.

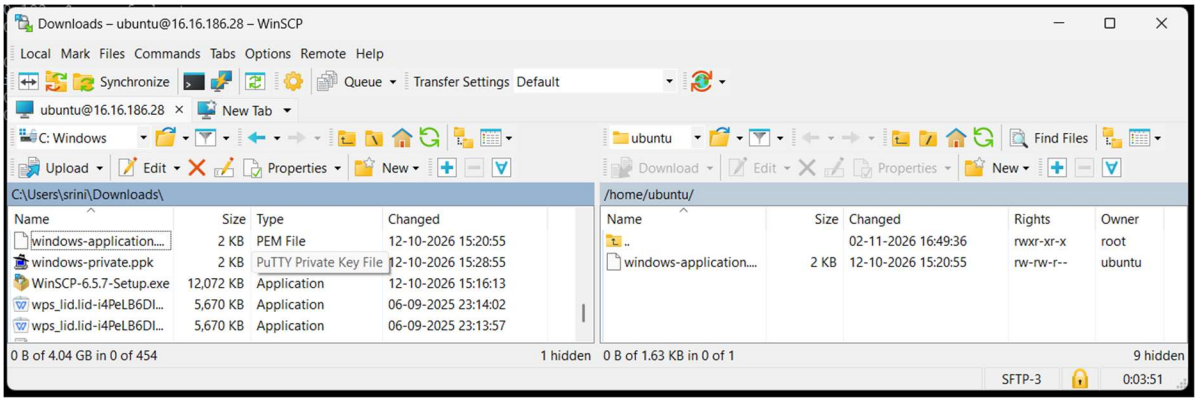


Figure 32. File transfer client view used during the lab.

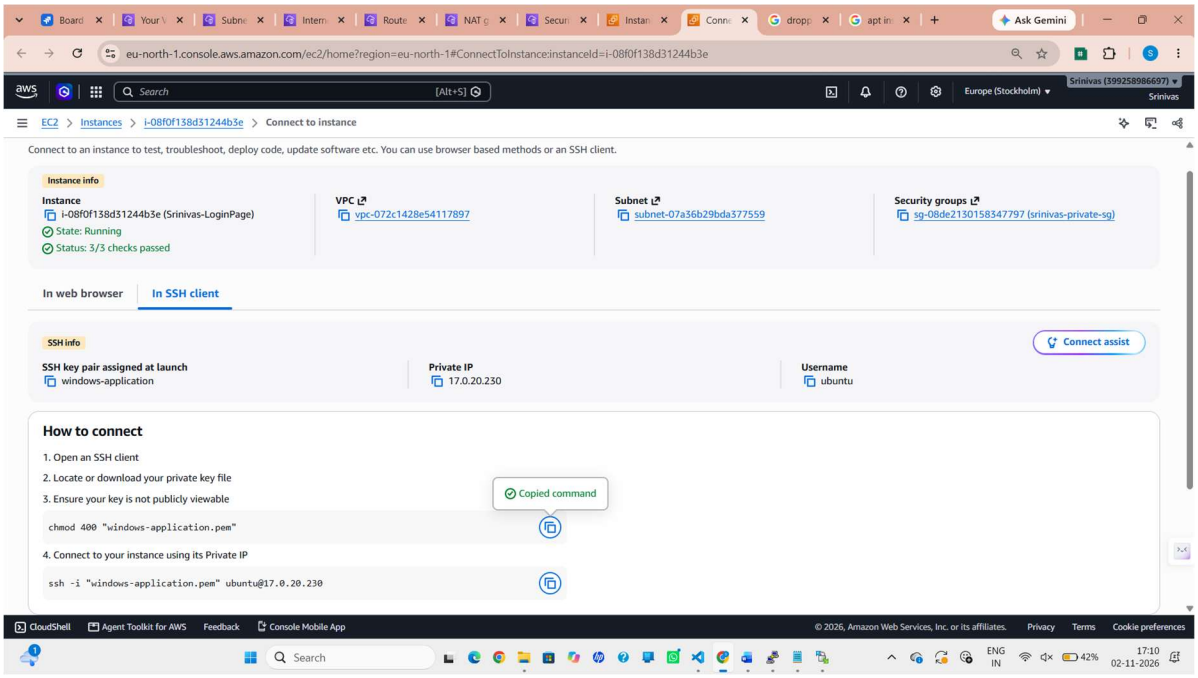


Figure 33. EC2 connection details in the console.

12. Connect to the private instance securely

Use the public EC2 instance as a jump host. Prefer SSH agent forwarding or AWS Systems Manager Session Manager where available. Do not store or copy an unprotected private key on the public host.

If using SSH agent forwarding from a local machine, load the key into your local SSH agent, then connect to the public instance with agent forwarding enabled. From the public instance, SSH to the private instance using its private IPv4 address. Ensure the private security group permits SSH from the public security group only.

The screenshots show PuTTY authentication settings and terminal connectivity. The exact key path and usernames depend on the AMI and local workstation.

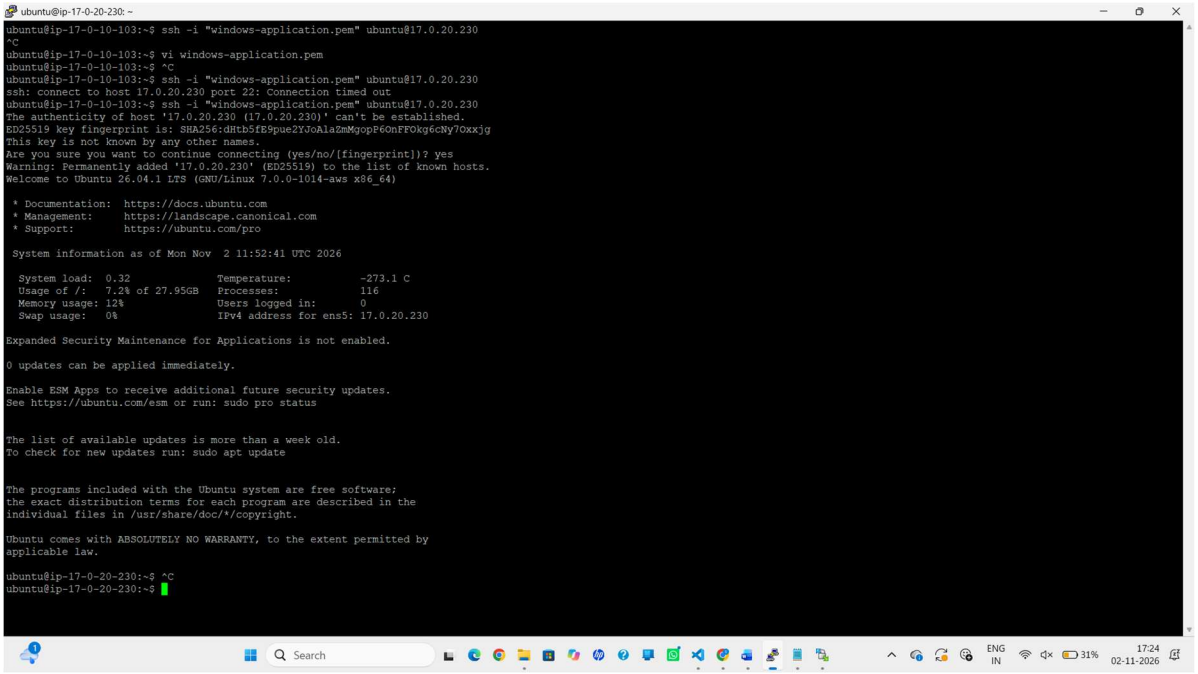


Figure 34. Terminal session showing package installation / server preparation.

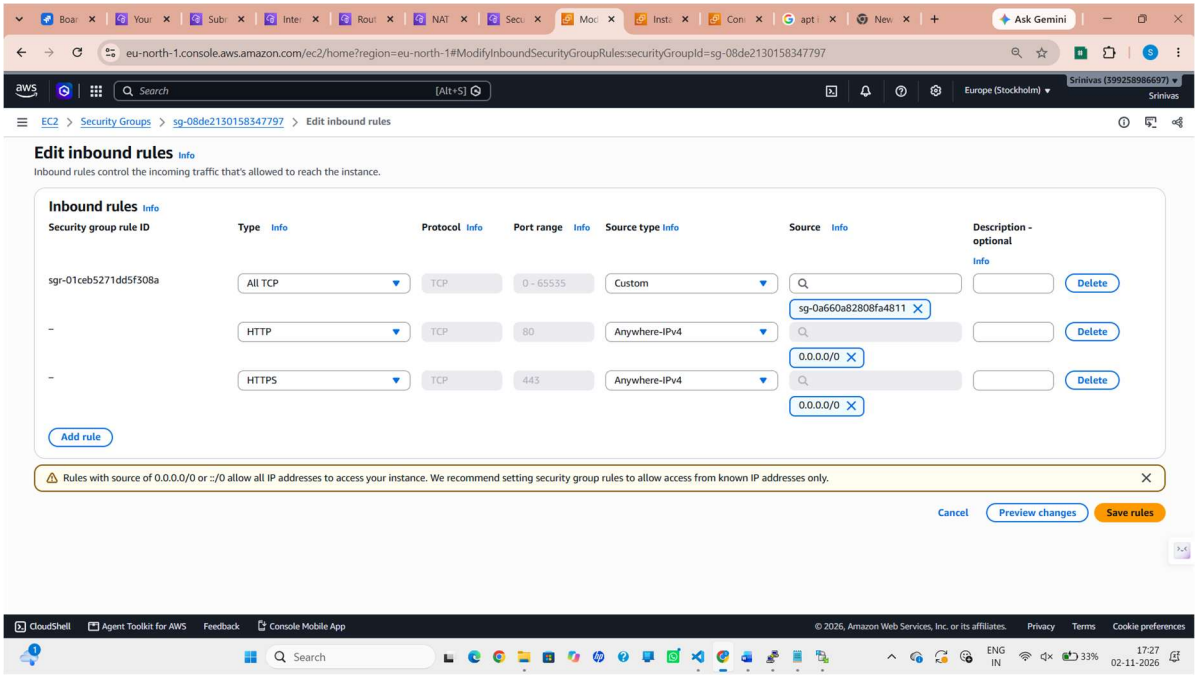


Figure 35. Security-group rule editing during connectivity setup.

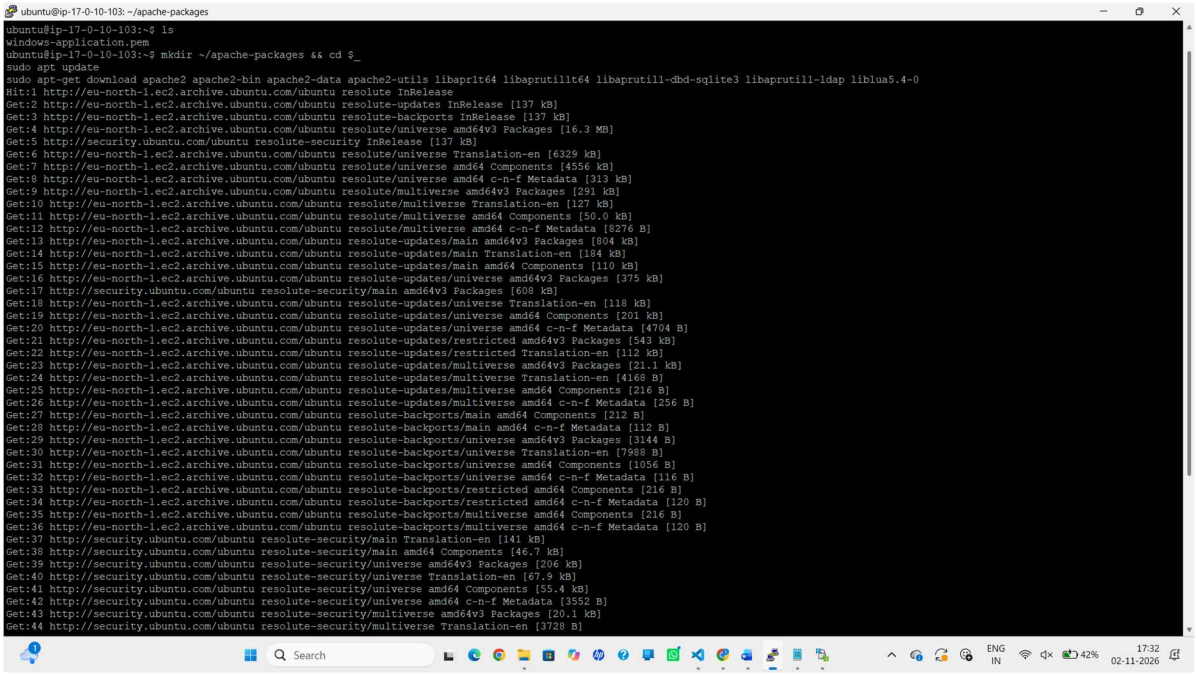


Figure 36. Terminal output during package installation.

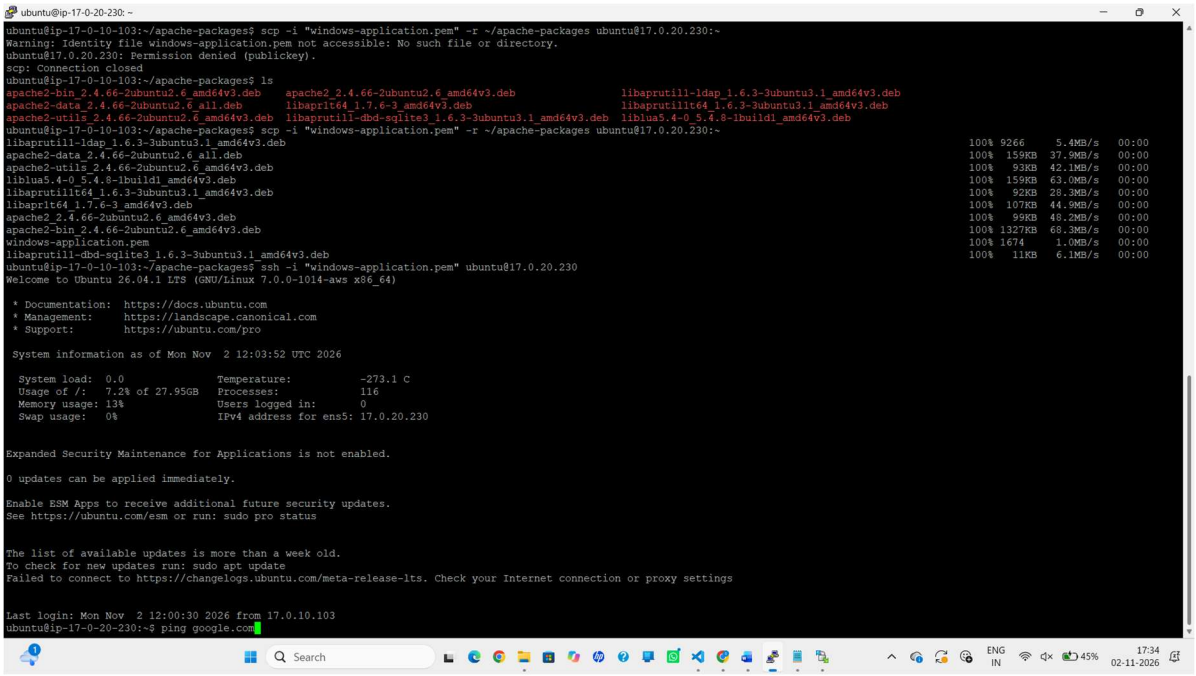


Figure 37. Terminal output and connectivity checks.

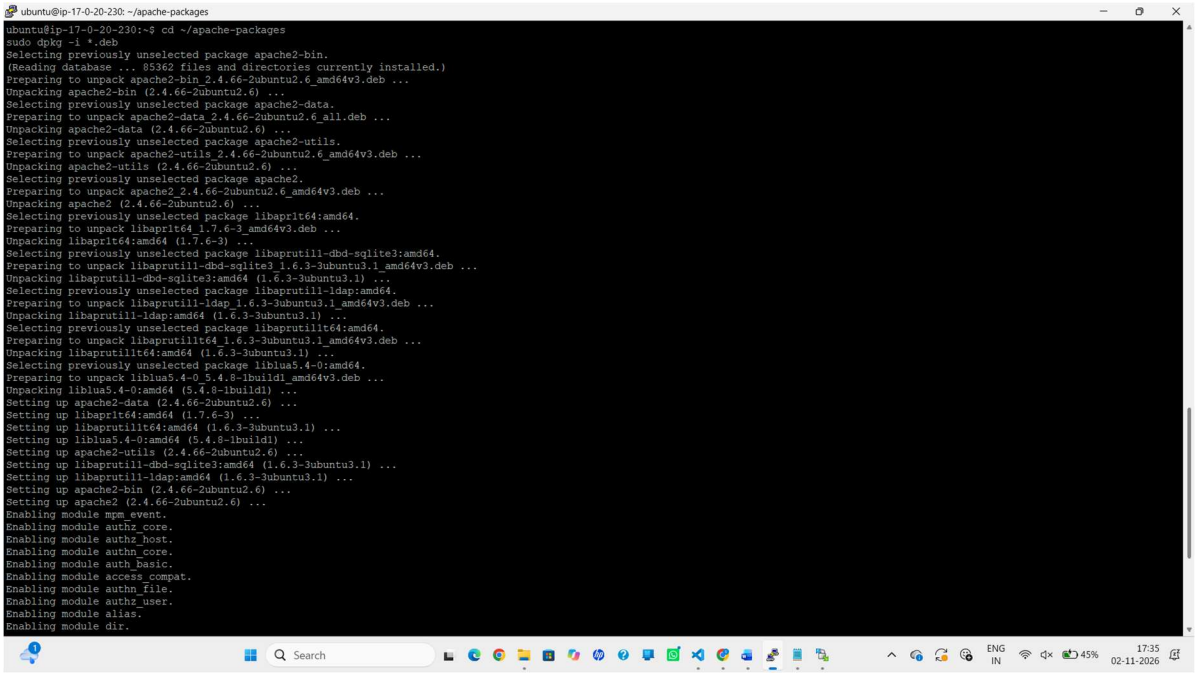


Figure 38. Terminal session after package operations.

13. Deploy the public Home Page

On the public EC2 instance, install Apache2 (Ubuntu) and create the page.

Ubuntu commands:

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable --now apache2
sudo systemctl status apache2
cd /var/www/html
sudo vi index.html
```

Create a simple Home Page in /var/www/html/index.html. Save the file, then verify that Apache is listening on port 80.

```
<!DOCTYPE html>
<html>
<head><title>Public Portal</title></head>
<body>
  <h1>Welcome to My Public Portal</h1>
  <p>This page is hosted on the public EC2 instance.</p>
  <a href="http://PRIVATE_EC2_PRIVATE_IP/">Go to Login Page</a>
</body>
</html>
```

Replace PRIVATE_EC2_PRIVATE_IP with the private instance's actual private IPv4 address. The link works only from a network location permitted to reach the private instance.

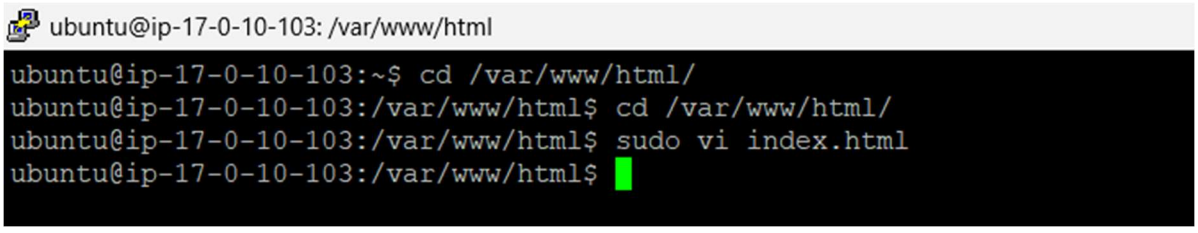


Figure 39. Home Page HTML being edited.

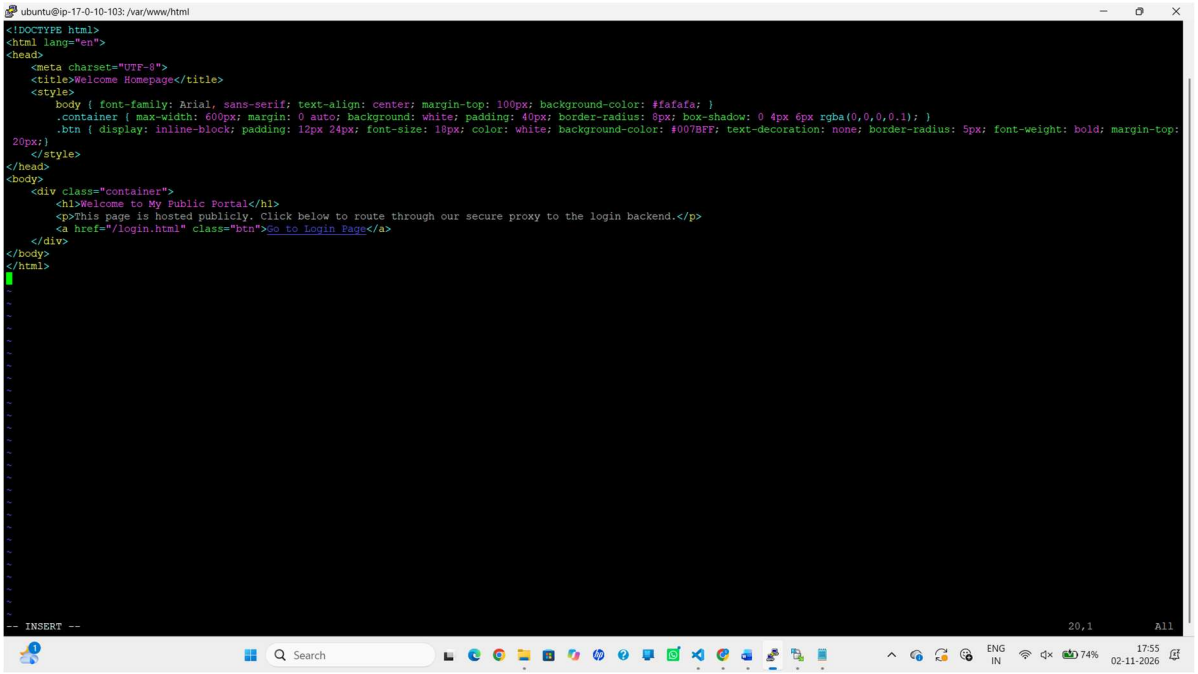


Figure 40. Updated Home Page HTML in the editor.

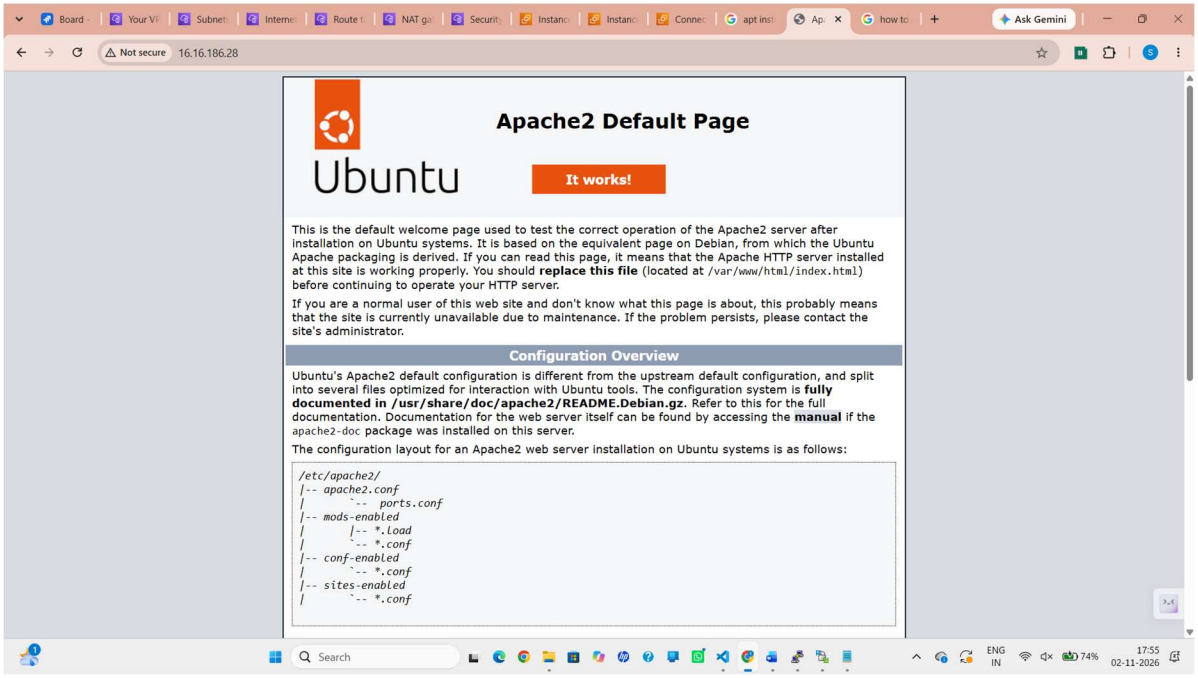


Figure 41. Apache2 default page before replacing the default content.

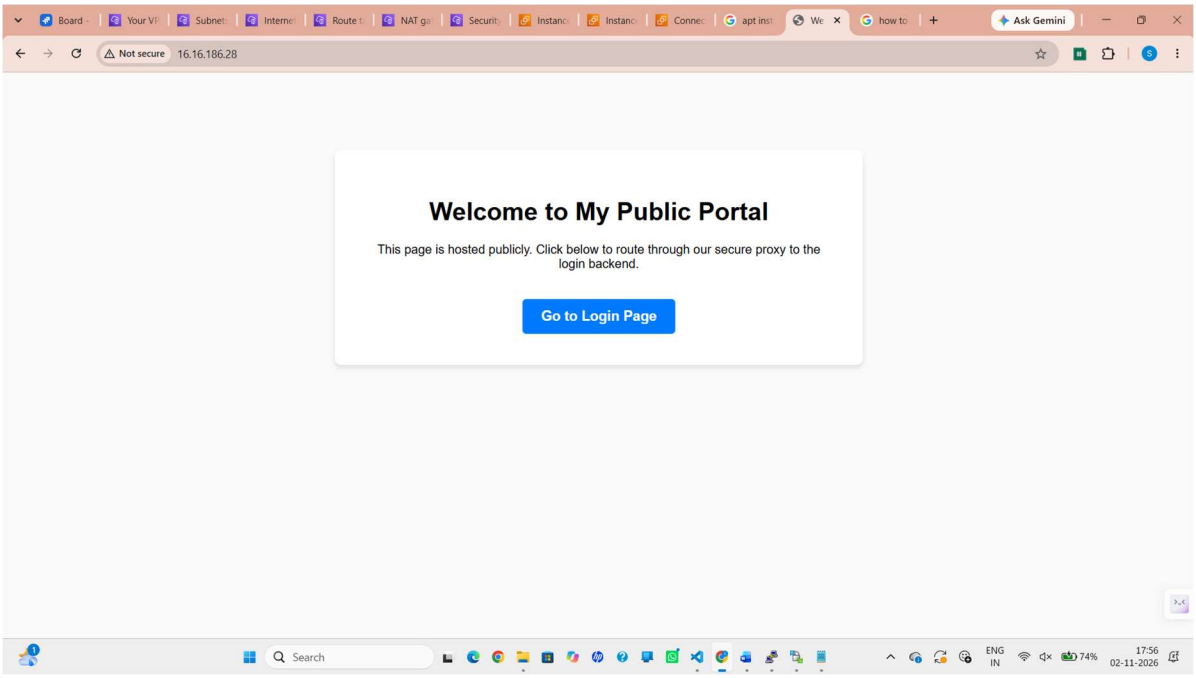


Figure 42. Custom public Home Page displayed in a browser.

14. Deploy the private Login Page

Connect to the private EC2 instance through the approved jump-host method. Install Apache2 and create a login page in /var/www/html/index.html.

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable --now apache2
sudo systemctl status apache2
cd /var/www/html
sudo vi index.html
```

Use a simple demonstration page. This is only a static HTML form; it does not authenticate users or securely process passwords.

```
<!DOCTYPE html>
<html>
<head><title>Secure Login</title></head>
<body>
  <h2>Secure Login</h2>
  <form>
    <input type="text" placeholder="Username">
    <input type="password" placeholder="Password">
    <button type="submit">Sign In</button>
  </form>
</body>
</html>
```

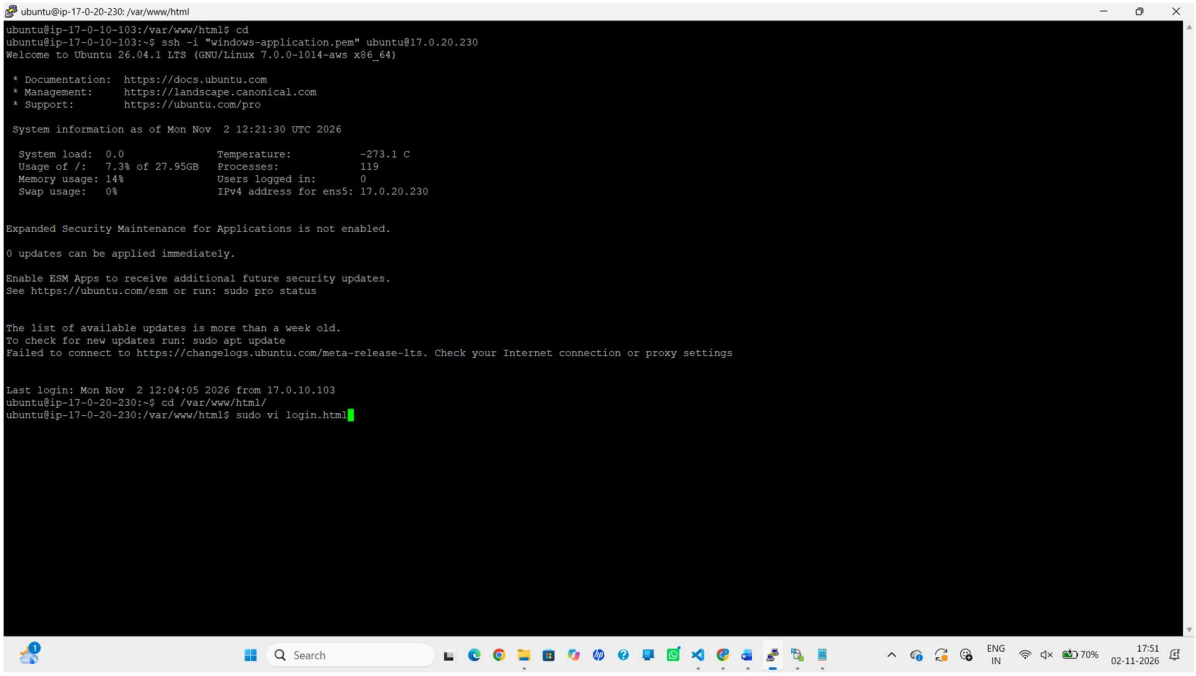


Figure 43. Terminal session used for web-server setup.

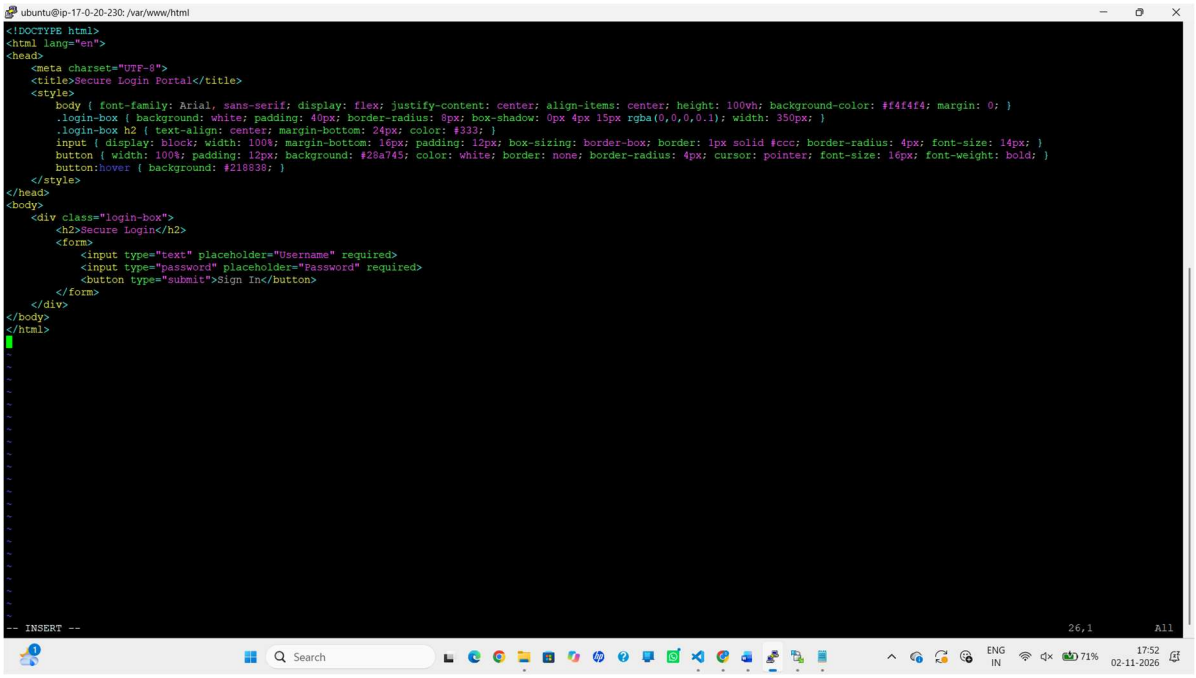


Figure 44. Editing the web page source in the terminal.

If Apache returns “Not Found”, check the document root, file name, and virtual-host configuration. The screenshots show a default-page / URL issue followed by editing the Apache default-site configuration and restarting Apache.

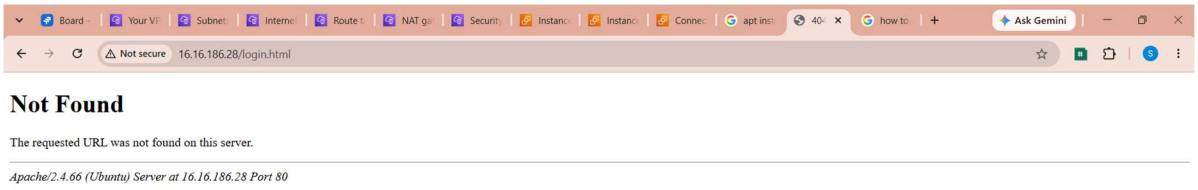


Figure 45. Browser showing a Not Found response during initial testing.



Figure 46. Apache default-site configuration being edited.

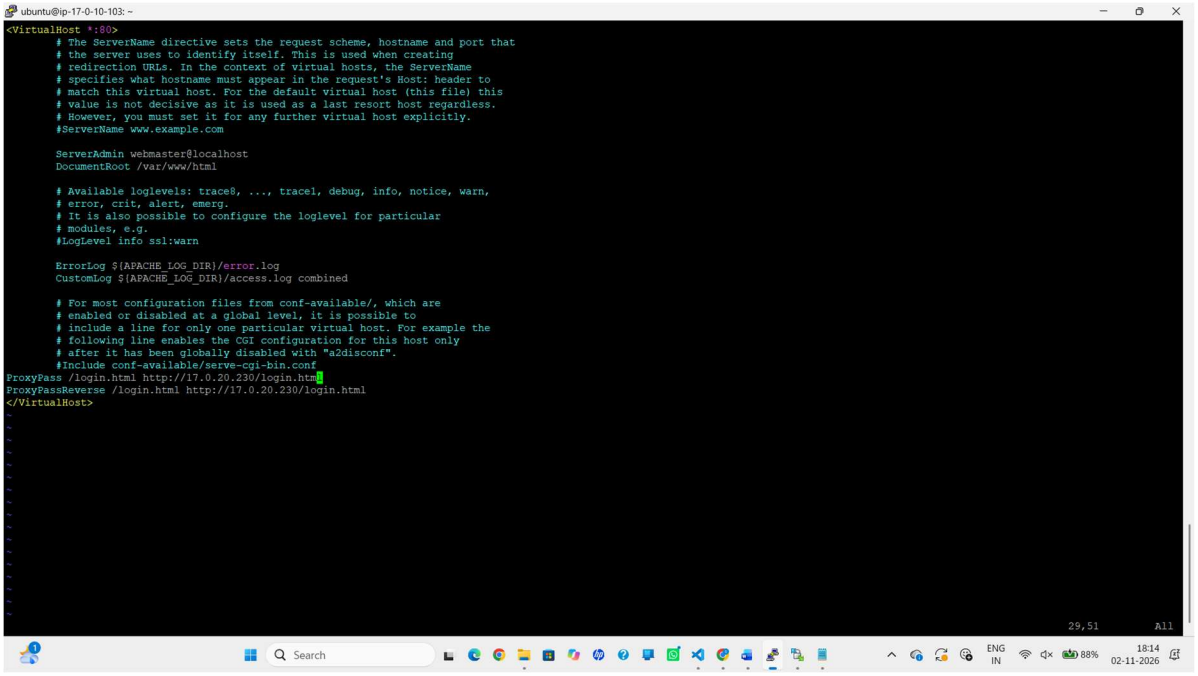


Figure 47. Apache service restart / configuration verification.

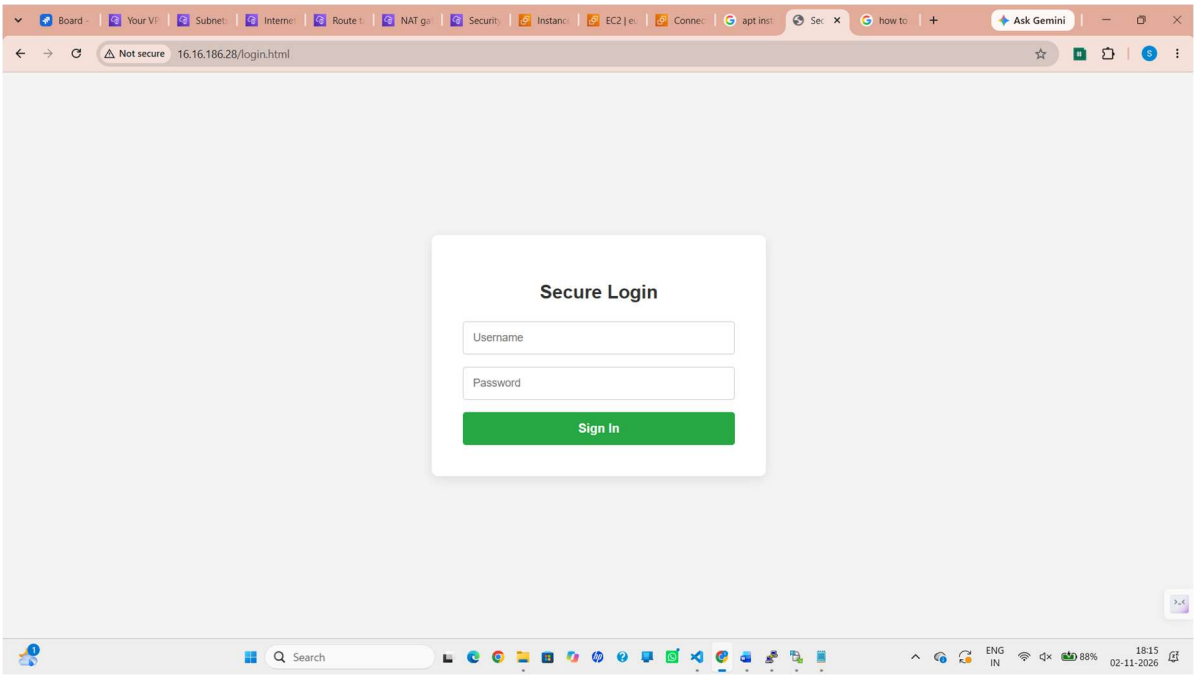


Figure 48. Private Login Page displayed in the browser.

15. Verify network connectivity

Perform the following checks after both web servers are running.

Check	Expected result	How to verify
Public EC2 has public IPv4	Public IPv4 address is assigned	EC2 → Instances → select public instance → Details
Home Page internet access	Page loads from an external browser	http://PUBLIC_EC2_PUBLIC_IP
Public SSH restriction	Inbound TCP 22 source is your IP /32	EC2 → Security Groups → public SG → Inbound rules
Public route	0.0.0.0/0 points to the Internet Gateway	VPC → Route Tables → public route table
Private EC2 public IPv4	No public IPv4 address	EC2 → Instances → select private instance → Details
Private page direct access	Not reachable from the public internet	Do not expose a public IP; test from outside the VPC
Public-to-private HTTP	Public EC2 can fetch private page	From public EC2: curl http://PRIVATE_EC2_PRIVATE_IP
Private route	0.0.0.0/0 points to NAT Gateway	VPC → Route Tables → private route table
Private outbound internet	Private EC2 can initiate outbound connection	From private EC2: curl -I https://example.com

Useful verification commands

```
# On the public EC2 instance
curl -I http://localhost
curl http://PRIVATE_EC2_PRIVATE_IP

# On the private EC2 instance
curl -I http://localhost
curl -I https://example.com
```

```
# Confirm Apache is listening
sudo ss -lntp | grep ':80'
```

Note: NAT Gateway supports outbound connections initiated from the private subnet. It does not allow an internet host to initiate a new connection to the private EC2 instance.

16. Cleanup and cost control

NAT Gateways and Elastic IP addresses can incur charges. When the lab is complete, remove resources that are no longer required. Before deleting anything, confirm that it belongs only to this lab and not to another workload.

33. Terminate the two lab EC2 instances.
34. Delete the NAT Gateway and release its Elastic IP after the NAT Gateway deletion completes, if the Elastic IP is no longer needed.
35. Detach and delete the lab Internet Gateway.
36. Delete the lab route tables after removing subnet associations (the main route table cannot be deleted).
37. Delete the lab security groups after they are no longer attached or referenced.
38. Delete the lab subnets, then delete the lab VPC.

17. Key learning outcomes

- A VPC provides an isolated IP network in AWS.
- Subnets divide the VPC address range into smaller network segments.
- An Internet Gateway provides a route between the VPC and the internet for public resources.
- A NAT Gateway allows private-subnet resources to initiate outbound internet connections.
- Route tables determine where subnet traffic is sent.
- Security groups act as stateful instance-level firewalls.
- A private EC2 instance can serve internal applications without a public IPv4 address.